INNOPOLIS UNIVERSITY

SNE MASTER
TRACK 2026

COURSE: [S26] DevOps and Security

LS lab 4 Report

On

Kubernetes Basics

Student Name: HASIB AL TAHSIN

Application ID: 187623
Student ID: 24

Email: h.tahsin@innopolis.university, hasibjust10@gmail.com

Submission Date: 19.02.2026

Index

Task 1…… 3-11

Task 2……... 12-17

Task 3……... 17-24

Task 4……...… 24-39

Task 5……... 49-56

Task 6…….…….. 57-66

Task 7……... 66-76

Task 8……...… 81-90

Task 9……..……. 90-97

Bonus Part…….……... 97-107

References…….. 108

Task 1 - Preparation

1.1. Choose an application that has such characteristics as (points with "star" are not required but recommended):

- **any non-confidential data like in JSON, XML or TOML format**
- **application configuration values (or environment variables)**
- **any secrets (like authorization credentials)**
- **availability from outside (on Internet)**
- **has any health checks or/and status endpoints**

It could be any yours application or open-source project that meets the requirements. You are even able to work with different applications in different lab tasks but try to avoid it. However, it's much better to work with the single project but it's acceptable to involve only the required application functionality regarding the particular task requirements. Put the git link to your chosen project into report and provide a brief project description for what it is and how it works.

1.2. Get familiar with Kubernetes (k8s) and concepts.

1.3. Install and set up the necessary tools: kubectl minikube or its alternative

1.4. Get access to Kubernetes Dashboard.

Solution:

1.1. Chosen Application:

I used Nginx in this lab because it-

- Can be deployed quickly as an HTTP server
- Suitable for service / DNS testing
- Logs/exec/scaling are all visible
- env vars, configmap, secret mount, rolling update, rollback are all doable

Project/Container Source: Docker Hub - nginx image

Application: nginx

Nginx is a lightweight web server. Running it as a Kubernetes Pod/Deployment allows you to view container logs, access pod shell, route traffic with services, and scale/rollout with deployments.

1.2. Kubernetes Concepts:

- **Cluster:** In Kubernetes, a cluster is a complete orchestration environment where our applications are deployed and managed. A cluster is basically made up of two parts - Control Plane and Worker Nodes. The Control Plane acts as the brain of the entire system and does scheduling, state management and decision-making. On the other hand, Worker Nodes actually run application containers. When *Minikube* is used, a single machine - this Control Plane and Worker Node - coexist. But in production environments, multiple control planes and many worker nodes are usually used to ensure high availability and scalability.
- **Node:** A node is the machine (virtual or physical) where Kubernetes application workloads run. Each node has an agent called a *kubelet*, which communicates with the control plane and ensures that specific Pods are running properly. In addition, the node has a container runtime, which takes care of running containers. *kube-proxy* handles networking and service routing. *Minikube* usually has one node, but a real production cluster can have many nodes depending on the workload.
- **Pod:** A pod is the smallest deployable unit of Kubernetes. We do not run containers directly; rather, containers always run inside a Pod. A Pod can contain one or more containers, and they share the same network namespace and storage. A Pod has its own IP address, but if a Pod is deleted or recreated, its IP can change. Therefore, a Pod is considered a temporary and dynamic component. In general, one Pod is used for each microservice.
- **Service:** Since Pods in Kubernetes are dynamic, their IP address does not remain permanent. A Service is used to solve this problem. A Service provides a stable IP address and DNS name, through which other Pods or external clients can consistently connect to a specific application. A Service also does load balancing, distributing traffic among multiple Pods of the same application. There are three main types of Services, *ClusterIP, NodePort,* and *LoadBalancer*, which are used according to different networking needs.
- **Deployment:** Deployment is a higher-level controller in Kubernetes, which manages the Pod lifecycle. It first creates a *ReplicaSet* instead of directly creating Pods, and that *ReplicaSet* creates and maintains Pods as needed. Deployment can be used to scale the application (e.g. from 1 Pod to 3 Pods), perform rolling updates (gradually releasing a new version), and rollback. Deployment is very important to ensure stable and zero-downtime updates in the production environment.
- ***kubectl:*** *kubectl* is a Command Line Interface (CLI) tool used to interact with the Kubernetes cluster. Through it, we can create, modify, delete, and monitor the resources of the cluster. Each command of *kubectl* basically communicates with the Kubernetes API Server and takes control plane action accordingly. We can monitor the current status of the cluster using commands such as *kubectl* get pods, *kubectl* describe pod, *kubectl* logs, etc. *kubectl* is an essential tool in DevOps workflow, as it is used in both automation and troubleshooting.

1.3. Install & Setup Necessary Tools:

Docker verification:

The Kubernetes cluster is run using the *Minikube* Docker driver. So first, verify that Docker is properly installed and running.

```
hasib@gitlab-24:~$ docker --version
```

This command verifies whether the Docker client is installed and which version is running. The output shows *Docker version 29.2.1*.

```
hasib@gitlab-24:~$ docker --version
Docker version 29.2.1, build a5c7197
```

The test container was run using the following command. Here Docker contacted the Docker daemon. It pulled the *hello-world* image from Docker Hub and created and ran a container.

```
hasib@gitlab-24:~$ docker --version
```

Seeing *"Hello from Docker!"* in the output confirms that the Docker daemon is working properly.

```
hasib@gitlab-24:~$ docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
 $ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
 https://hub.docker.com/

For more examples and ideas, visit:
 https://docs.docker.com/get-started/
```

Troubleshooting - permission error:

This adds the user to the docker group, so that docker commands can be used without *sudo*.

```
hasib@gitlab-24:~$ sudo usermod -aG docker $USER
```

```
hasib@gitlab-24:~$ newgrp docker
```

Installing kubectl:

kubectl is a CLI tool for communicating with Kubernetes clusters. The official binary method was used due to the classic confinement issue in Snap install.

```
hasib@gitlab-24:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
```

This command downloads the latest stable version of Kubernetes. It fetches the binary file directly from the official Kubernetes source.

```
hasib@gitlab-24:~$ curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100 55.8M  100 55.8M    0     0  24.4M      0  0:00:02  0:00:02 --:--:-- 24.4M
```

The following command installs the *kubectl* binary to the system-wide executable path */usr/local/bin*. Now the *kubectl* command can be used from any directory.

```
hasib@gitlab-24:~$ sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

The *kubectl* client version has been verified using the following command.

```
hasib@gitlab-24:~$ kubectl version --client --output=yaml
```

The output shows *gitVersion: v1.35.1*, indicating that *kubectl* was successfully installed.

```
hasib@gitlab-24:~$ kubectl version --client --output=yaml
clientVersion:
  buildDate: "2026-02-10T12:55:59Z"
  compiler: gc
  gitCommit: 8fea90b45245ef5c8ba54e7ae044d3e777c22500
  gitTreeState: clean
  gitVersion: v1.35.1
  goVersion: go1.25.6
  major: "1"
  minor: "35"
  platform: linux/amd64
kustomizeVersion: v5.7.1
```

Install minikube (Docker driver):

Here a local Kubernetes cluster has been created using *Minikube*.

```
hasib@gitlab-24:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

The above command downloaded the latest *Minikube* binary, where the file size was 128MB and the download was successful.

```
hasib@gitlab-24:~$ curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
  % Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
                                 Dload  Upload   Total   Spent    Left  Speed
100  128M  100  128M    0     0  19.5M      0  0:00:06  0:00:06 --:--:-- 24.2M
```

The following command has installed the *Minikube* binary to the system path.

```
hasib@gitlab-24:~$ sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Here it can be seen that *Minikube version v1.38.0* has been successfully installed.

```
hasib@gitlab-24:~$ minikube version
minikube version: v1.38.0
commit: de81223c61ab1bd97dcfcfa6d9d5c59e5da4a0cf
```

Starting local cluster

The following command creates a local Kubernetes cluster using the Docker driver.

```
hasib@gitlab-24:~$ minikube start --driver=docker
```

In this process, the Base image has been pulled and the Kubernetes preload image has been downloaded. As a result, the Docker container has been created and the Control-plane has been initialized.

```
hasib@gitlab-24:~$ minikube start --driver=docker
😄  minikube v1.38.0 on Ubuntu 24.04
✨  Using the docker driver based on user configuration
❗  Starting v1.39.0, minikube will default to "containerd" container runtime. See #21973 for more info.
📌  Using Docker driver with root privileges
👍  Starting "minikube" primary control-plane node in "minikube" cluster
🚜  Pulling base image v0.0.49 ...
💾  Downloading Kubernetes v1.35.0 preload ...
    > preloaded-images-k8s-v18-v1...:  271.45 MiB / 271.45 MiB  100.00% 18.55 M
    > gcr.io/k8s-minikube/kicbase...:  514.15 MiB / 514.16 MiB  100.00% 15.50 M

🔥  Creating docker container (CPUs=2, Memory=3072MB) ...
🐳  Preparing Kubernetes v1.35.0 on Docker 29.2.0 ...
🔗  Configuring bridge CNI (Container Networking Interface) ...
🔎  Verifying Kubernetes components...
    ▪ Using image gcr.io/k8s-minikube/storage-provisioner:v5
🌟  Enabled addons: storage-provisioner, default-storageclass
🏄  Done! kubectl is now configured to use "minikube" cluster and "default" namespace by default
```

The output above shows that *kubectl* has automatically joined this cluster.

Running the command below shows that the output shows that Node status is Ready, meaning the cluster is running successfully.

```
hasib@gitlab-24:~$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   63s   v1.35.0
```

Enabling Metrics Server:

The metrics-server has been enabled to view CPU and Memory usage by running the following command.

```
hasib@gitlab-24:~$ minikube addons enable metrics-server
💡  metrics-server is an addon maintained by Kubernetes. For any concerns contact minikube on GitHub.
You can view the list of minikube maintainers at: https://github.com/kubernetes/minikube/blob/master/OWNERS
    ▪ Using image registry.k8s.io/metrics-server/metrics-server:v0.8.0
🌟  The 'metrics-server' addon is enabled
```

Verifying Pod:

```
hasib@gitlab-24:~$ kubectl get pods -n kube-system | grep metrics
```

The output below shows that the metrics-server pod is running successfully.

```
hasib@gitlab-24:~$ kubectl get pods -n kube-system | grep metrics
metrics-server-5778bb4788-xlsgt    1/1     Running   1 (2m35s ago)   13m
```

Verifying API:

```
hasib@gitlab-24:~$ kubectl get apiservice v1beta1.metrics.k8s.io -o wide
```

The output from the above command shows that the metrics API is active.

```
hasib@gitlab-24:~$ kubectl get apiservice v1beta1.metrics.k8s.io -o wide
NAME                     SERVICE                      AVAILABLE   AGE
v1beta1.metrics.k8s.io   kube-system/metrics-server   True        13m
```

Checking Resource Usage:

```
hasib@gitlab-24:~$ kubectl top nodes
```

The above command shows CPU and Memory usage. The output below confirms that metrics-server is working properly. The metrics-server pod was Running.

```
hasib@gitlab-24:~$ kubectl top nodes
NAME       CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube   340m         8%       957Mi           12%
hasib@gitlab-24:~$ mkdir -p ~/Desktop/k8s-lab-diamond
```

To resolve metrics not available issue:

The following command was used to resolve metrics not available issue.

```
hasib@gitlab-24:~$ kubectl logs -n kube-system deploy/metrics-server --tail=80
I0215 01:48:13.795682       1 handler.go:288] Adding GroupVersion metrics.k8s.io v1beta1 to ResourceManager
I0215 01:48:14.452790       1 secure_serving.go:211] Serving securely on [::]:4443
I0215 01:48:14.452933       1 requestheader_controller.go:180] Starting RequestHeaderAuthRequestController
I0215 01:48:14.452955       1 shared_informer.go:350] "Waiting for caches to sync" controller="RequestHeaderAuthRequestController"
I0215 01:48:14.452991       1 dynamic_serving_content.go:135] "Starting controller" name="serving-cert::/tmp/apiserver.crt::/tmp/apiserver.key"
I0215 01:48:14.453235       1 tlsconfig.go:243] "Starting DynamicServingCertificateController"
I0215 01:48:14.467803       1 configmap_cafile_content.go:205] "Starting controller" name="client-ca::kube-system::extension-apiserver-authentication::requestheader-client-ca-file"
I0215 01:48:14.467864       1 shared_informer.go:350] "Waiting for caches to sync" controller="client-ca::kube-system::extension-apiserver-authentication::requestheader-client-ca-file"
I0215 01:48:14.460718       1 configmap_cafile_content.go:205] "Starting controller" name="client-ca::kube-system::extension-apiserver-authentication::client-ca-file"
I0215 01:48:14.475329       1 shared_informer.go:350] "Waiting for caches to sync" controller="client-ca::kube-system::extension-apiserver-authentication::client-ca-file"
I0215 01:48:14.553334       1 shared_informer.go:357] "Caches are synced" controller="RequestHeaderAuthRequestController"
I0215 01:48:14.568802       1 shared_informer.go:357] "Caches are synced" controller="client-ca::kube-system::extension-apiserver-authentication::requestheader-client-ca-file"
I0215 01:48:14.576191       1 shared_informer.go:357] "Caches are synced" controller="client-ca::kube-system::extension-apiserver-authentication::client-ca-file"
```

Then, if I run kubectl top nodes again, the metrics are visible.

```
hasib@gitlab-24:~$ kubectl top nodes
NAME       CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube   340m         8%       957Mi           12%
hasib@gitlab-24:~$ mkdir -p ~/Desktop/k8s-lab-diamond
```

In this case, this patch is required to make kubectl top nodes work (common in minikube env):

```
hasib@gitlab-24:~$ kubectl -n kube-system patch deployment metrics-server \
  --type='json' \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"},{"op":"add","path":"/spec/template/spec/containers/0/args/-",
"value":"--kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname"}]'
```

Deployment.apps/metrics-server patched was found in the output.

```
hasib@gitlab-24:~$ kubectl -n kube-system patch deployment metrics-server \
  --type='json' \
  -p='[{"op":"add","path":"/spec/template/spec/containers/0/args/-","value":"--kubelet-insecure-tls"},{"op":"add","path":"/spec/template/spec/containers/0/args/-",
"value":"--kubelet-preferred-address-types=InternalIP,ExternalIP,Hostname"}]'
deployment.apps/metrics-server patched
```

Rollout:

```
hasib@gitlab-24:~$ kubectl rollout status -n kube-system deploy/metrics-server
```

Using the above output, the rollout was successful.

```
hasib@gitlab-24:~$ kubectl rollout status -n kube-system deploy/metrics-server
Waiting for deployment "metrics-server" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "metrics-server" rollout to finish: 1 old replicas are pending termination...
deployment "metrics-server" successfully rolled out
```

Then, using the following command, the CPU and Memory numbers were verified.

```
hasib@gitlab-24:~$ kubectl top nodes
NAME       CPU(cores)   CPU(%)   MEMORY(bytes)   MEMORY(%)
minikube   340m         8%       957Mi           12%
```

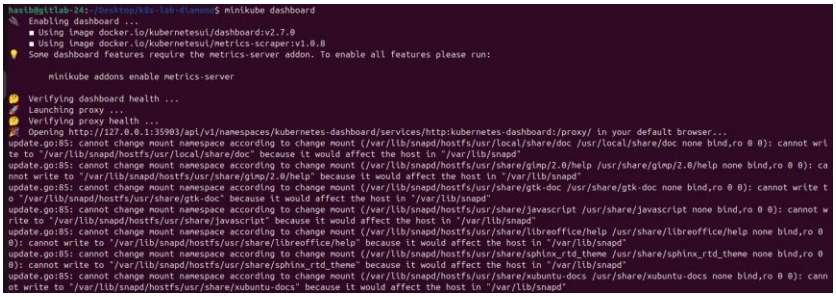
1.4. Kubernetes Dashboard Access:

Kubernetes Dashboard is used to monitor and manage Kubernetes cluster from GUI environment. Dashboard addon is enabled in *Minikube* environment and accessed through browser through local proxy. Dashboard is Kubernetes web-based UI, through which Pods, Deployments, Services, Nodes etc. can be visually viewed and managed.

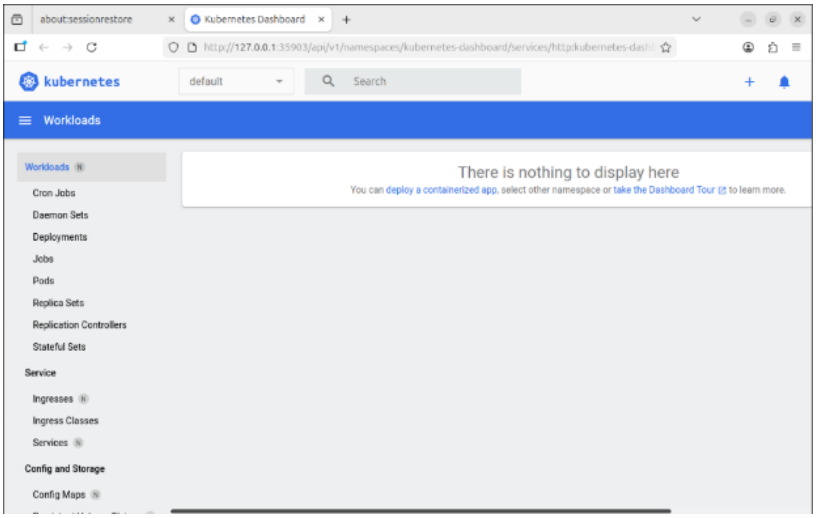
Kubernetes Dashboard is launched with the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ minikube dashboard
```

By executing the above command, *Minikube* Dashboard addon is enabled, so that Kubernetes Dashboard pod is deployed to the cluster. Then Dashboard image is pulled so that required container images are downloaded. Then *Minikube* creates a secure local proxy and generates Local URL. Dashboard is successfully run and access URL is generated in local browser.



The dashboard URL opened correctly:



Node OS + CPU + Kernel proof:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes -o wide
```

output:

version: 1.35.0 , OS: Debian GNU/Linux 12 (bookworm), Kernel version: 6.17.0-14-generic

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION      CONTAINER-RUNTIME
minikube   Ready    control-plane   27m   v1.35.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   6.17.0-14-generic   docker://29.2.0
```

Describe Node:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe node minikube
Name:               minikube
Roles:              control-plane
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=minikube
                    kubernetes.io/os=linux
                    minikube.k8s.io/commit=de81223c61ab1bd97dcfcfa6d9d5c59e5da4a0cf
                    minikube.k8s.io/name=minikube
                    minikube.k8s.io/primary=true
                    minikube.k8s.io/updated_at=2026_02_15T04_36_01_0700
                    minikube.k8s.io/version=v1.38.0
                    node-role.kubernetes.io/control-plane=
                    node.kubernetes.io/exclude-from-external-load-balancers=
Annotations:        node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp:  Sun, 15 Feb 2026 04:35:57 +0300
Taints:             <none>
Unschedulable:      false
Lease:
  HolderIdentity:  minikube
  AcquireTime:     <unset>
  RenewTime:       Sun, 15 Feb 2026 05:03:32 +0300
```

```
Taints:             <none>
Unschedulable:      false
Lease:
  HolderIdentity:  minikube
  AcquireTime:     <unset>
  RenewTime:       Sun, 15 Feb 2026 05:03:32 +0300
Conditions:
  Type             Status  LastHeartbeatTime                 LastTransitionTime                Reason                       Message
  ----             ------  -----------------                 ------------------                ------                       -------
  MemoryPressure   False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasSufficientMemory   kubelet has sufficient memory available
  DiskPressure     False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            True    Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:48:09 +0300   KubeletReady                 kubelet is posting ready status
Addresses:
  InternalIP:  192.168.49.2
  Hostname:    minikube
Capacity:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
Allocatable:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
```

```
  System UUID:                f5a84ec7-d405-4bb3-b014-eafd8ea5b392
  Boot ID:                    ccb1c286-fd97-4e7a-a4c2-7cdebcd9a76a
  Kernel Version:             6.17.0-14-generic
  OS Image:                   Debian GNU/Linux 12 (bookworm)
  Operating System:           linux
  Architecture:               amd64
  Container Runtime Version:  docker://29.2.0
  Kubelet Version:            v1.35.0
  Kube-Proxy Version:
PodCIDR:                      10.244.0.0/24
PodCIDRs:                     10.244.0.0/24
Non-terminated Pods:          (10 in total)
  Namespace                   Name                                         CPU Requests  CPU Limits  Memory Requests  Memory Limits  Age
  ---------                   ----                                         ------------  ----------  ---------------  -------------  ---
  kube-system                 coredns-7d764666f9-t8s2q                     100m (2%)     0 (0%)      70Mi (0%)        170Mi (2%)     27m
  kube-system                 etcd-minikube                                100m (2%)     0 (0%)      100Mi (1%)       0 (0%)         27m
  kube-system                 kube-apiserver-minikube                      250m (6%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-controller-manager-minikube             200m (5%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-proxy-62lwk                             0 (0%)        0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-scheduler-minikube                      100m (2%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 metrics-server-855945dcdd-xkdsx              100m (2%)     0 (0%)      200Mi (2%)       0 (0%)         12m
  kube-system                 storage-provisioner                          0 (0%)        0 (0%)      0 (0%)           0 (0%)         27m
  kubernetes-dashboard        dashboard-metrics-scraper-5565989548-lkd7l   0 (0%)        0 (0%)      0 (0%)           0 (0%)         10m
  kubernetes-dashboard        kubernetes-dashboard-b84665fb8-dgw8t         0 (0%)        0 (0%)      0 (0%)           0 (0%)         10m
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
```

```
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests    Limits
  --------           --------    ------
  cpu                850m (21%)  0 (0%)
  memory             370Mi (4%)  170Mi (2%)
  ephemeral-storage  0 (0%)      0 (0%)
  hugepages-1Gi      0 (0%)      0 (0%)
  hugepages-2Mi      0 (0%)      0 (0%)
Events:
  Type    Reason          Age   From             Message
  ----    ------          ----  ----             -------
  Normal  RegisteredNode  27m   node-controller  Node minikube event: Registered Node minikube in Controller
  Normal  RegisteredNode  15m   node-controller  Node minikube event: Registered Node minikube in Controller
```

Task 2 - k8s Nodes

We are going to start to learn Kubernetes from kubectl command line that is dedicated to work with k8s cluster.

1. After installing all required lab tools and starting minikube cluster, use kubectl commands to get and describe your cluster nodes.

2. Get the more detailed information about the particular node (it's fine if you have one node in the cluster).

3. Get the OS and CPU information.

4. Put the results into report.

Solution:

2.1 Using kubectl commands to get and describe cluster nodes:

After successfully starting the *Minikube* cluster, the cluster nodes were verified using *kubectl.*

Get Cluster Nodes:

The following command shows how many nodes are in the cluster and what their status is.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes
```

Analyzing the output below confirms that the cluster is running properly. Here, *minikube* is found as the Node Name, which indicates a local single-node cluster. Status Ready shows that the node is completely healthy and ready to run workloads (Pods). The role control-plane means that the same node is acting as the control-plane, which is normal for a *Minikube* single-node setup. The running version of the cluster is confirmed to be *Kubernetes Version v1.35.0* and *Age 36h* shows that the node has been running for about 36 hours. This proves that the cluster is properly running and operational.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   36h   v1.35.0
```

Wide view:

The following command was used to show the Node's OS, kernel, runtime, and IP details as strong proof in the report.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes -o wide
```

This output provides the full environment details of the cluster directly. *INTERNAL-IP = 192.168.49.2* confirms the cluster's internal networking IP, which will be very helpful when troubleshooting pods or services. *EXTERNAL-IP = <none>* shows that this is a local *Minikube* cluster, so there is no public external IP assigned, which is normal for a local development setup. *OS-IMAGE = Debian GNU/Linux 12 (bookworm) confirms* the base operating system of the node or VM. *KERNEL-VERSION = 6.17.0-14-generic* also confirms the kernel version, which is important for compatibility and troubleshooting. Finally, *CONTAINER-RUNTIME = docker://29.2.0* confirms that Docker is being used as the container runtime, which matches our Docker driver prerequisites perfectly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes -o wide
NAME       STATUS   ROLES           AGE   VERSION   INTERNAL-IP    EXTERNAL-IP   OS-IMAGE                         KERNEL-VERSION      CONTAINER-RUNTIME
minikube   Ready    control-plane   27m   v1.35.0   192.168.49.2   <none>        Debian GNU/Linux 12 (bookworm)   6.17.0-14-generic   docker://29.2.0
```

Describing node:

The following command shows the full configuration and health details of the node.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe node minikube
```

Important information related to the cluster environment has been successfully collected and verified from the output below. Here, the Internal IP of the Node has been found, which is necessary for internal networking and troubleshooting. In addition, the resource capability of the cluster has been confirmed by looking at the Total CPU and Memory capacity, that is, how much workload can be run safely. In addition, the base operating system of the node has been verified from the OS information, and the Kernel version has been seen to get an idea about system compatibility and stability. The container runtime version confirms which runtime is being used, which is important for deployment and container management. Finally, the hardware compatibility has been confirmed from the Architecture (e.g. amd64). This information has helped to completely verify the cluster environment and proved that the setup is configured correctly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe node minikube
Name:               minikube
Roles:              control-plane
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=minikube
                    kubernetes.io/os=linux
                    minikube.k8s.io/commit=de81223c61ab1bd97dcfcfa6d9d5c59e5da4a0cf
                    minikube.k8s.io/name=minikube
                    minikube.k8s.io/primary=true
                    minikube.k8s.io/updated_at=2026_02_15T04_36_01_0700
                    minikube.k8s.io/version=v1.38.0
                    node-role.kubernetes.io/control-plane=
                    node.kubernetes.io/exclude-from-external-load-balancers=
Annotations:        node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp:  Sun, 15 Feb 2026 04:35:57 +0300
Taints:             <none>
Unschedulable:      false
Lease:
  HolderIdentity:  minikube
  AcquireTime:     <unset>
  RenewTime:       Sun, 15 Feb 2026 05:03:32 +0300
```

```
Conditions:
  Type             Status  LastHeartbeatTime                 LastTransitionTime                Reason                       Message
  ----             ------  -----------------                 ------------------                ------                       -------
  MemoryPressure   False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasSufficientMemory   kubelet has sufficient memory available
  DiskPressure     False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            True    Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:48:09 +0300   KubeletReady                 kubelet is posting ready status
Addresses:
  InternalIP:  192.168.49.2
  Hostname:    minikube
Capacity:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
Allocatable:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
```

```
System Info:
  Machine ID:                 4571ab41f5a9a03217021e966978f901
  System UUID:                f5a84ec7-d405-4bb3-b014-eafd8ea5b392
  Boot ID:                    ccb1c286-fd97-4e7a-a4c2-7cdebcd9a76a
  Kernel Version:             6.17.0-14-generic
  OS Image:                   Debian GNU/Linux 12 (bookworm)
  Operating System:           linux
  Architecture:               amd64
  Container Runtime Version:  docker://29.2.0
  Kubelet Version:            v1.35.0
  Kube-Proxy Version:
PodCIDR:                      10.244.0.0/24
PodCIDRs:                     10.244.0.0/24
Non-terminated Pods:          (10 in total)
  Namespace                   Name                                         CPU Requests  CPU Limits  Memory Requests  Memory Limits  Age
  ---------                   ----                                         ------------  ----------  ---------------  -------------  ---
  kube-system                 coredns-7d764666f9-t8s2q                     100m (2%)     0 (0%)      70Mi (0%)        170Mi (2%)     27m
  kube-system                 etcd-minikube                                100m (2%)     0 (0%)      100Mi (1%)       0 (0%)         27m
  kube-system                 kube-apiserver-minikube                      250m (6%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-controller-manager-minikube             200m (5%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-proxy-62lwk                             0 (0%)        0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-scheduler-minikube                      100m (2%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 metrics-server-855945dcdd-xkdsx               100m (2%)     0 (0%)      200Mi (2%)       0 (0%)         12m
  kube-system                 storage-provisioner                          0 (0%)        0 (0%)      0 (0%)           0 (0%)         27m
  kubernetes-dashboard        dashboard-metrics-scraper-5565989548-lkd7l   0 (0%)        0 (0%)      0 (0%)           0 (0%)         10m
  kubernetes-dashboard        kubernetes-dashboard-b84665fb8-dgw8t         0 (0%)        0 (0%)      0 (0%)           0 (0%)         10m
```

```
Allocated resources:
  (Total limits may be over 100 percent, i.e., overcommitted.)
  Resource           Requests    Limits
  --------           --------    ------
  cpu                850m (21%)  0 (0%)
  memory             370Mi (4%)  170Mi (2%)
  ephemeral-storage  0 (0%)      0 (0%)
  hugepages-1Gi      0 (0%)      0 (0%)
  hugepages-2Mi      0 (0%)      0 (0%)
Events:
  Type    Reason          Age   From             Message
  ----    ------          ----  ----             -------
  Normal  RegisteredNode  27m   node-controller  Node minikube event: Registered Node minikube in Controller
  Normal  RegisteredNode  15m   node-controller  Node minikube event: Registered Node minikube in Controller
```

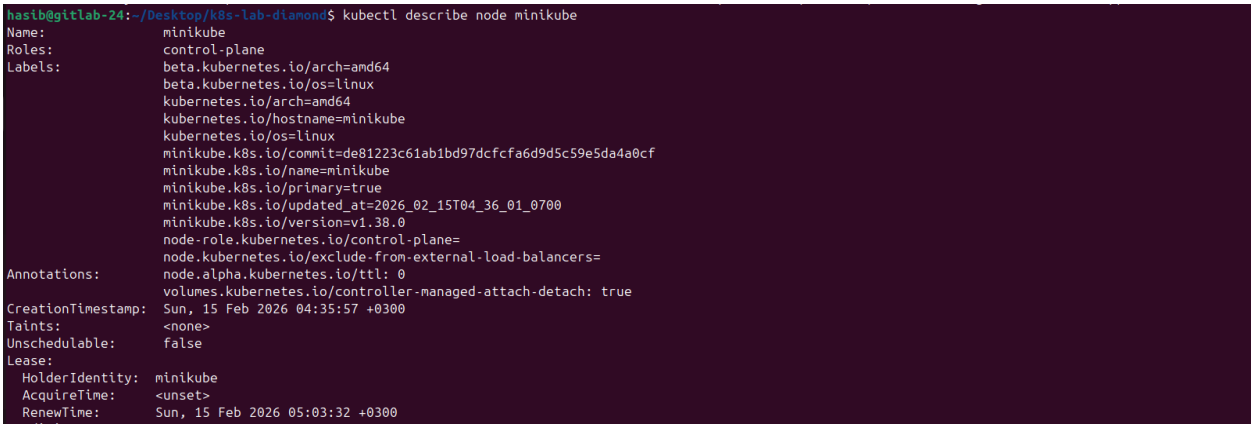
2.2. Getting the more detailed information about the particular node:

There is only one node (*minikube*) in the cluster, so the following command is used to view the detailed configuration of that node.

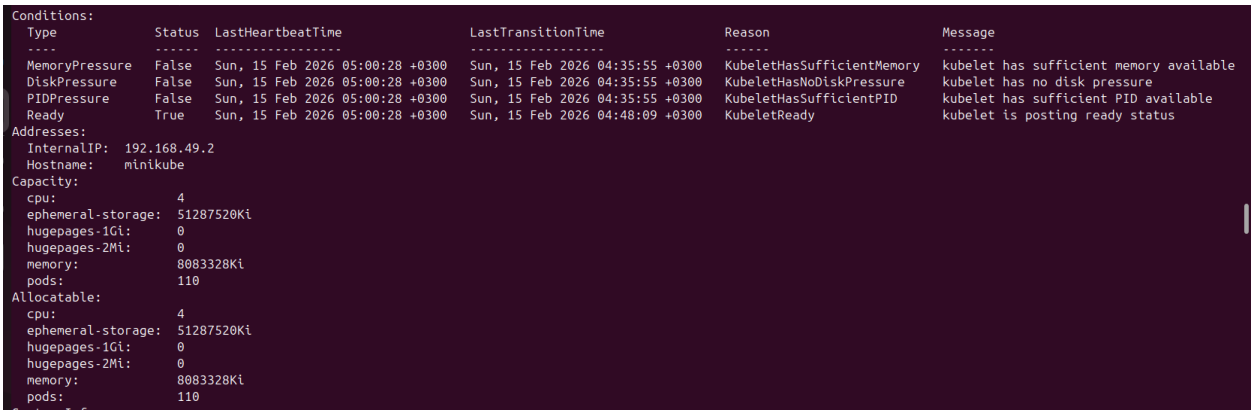
```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe node minikube
```

The following command successfully verified the node's complete technical environment. The Resource Capacity section shows that the node has 4 *CPU cores,* approximately *8083328Ki* (~7.7 GiB) of *Memory*, and the ability to run a maximum of 110 Pods, which is important for understanding the workload capacity of the cluster. The Network Information section confirms the Internal *IP 192.168.49.2,* which is necessary for internal communication and troubleshooting. The System Information section shows the *OS Image Debian GNU/Linux 12 (bookworm), Kernel*

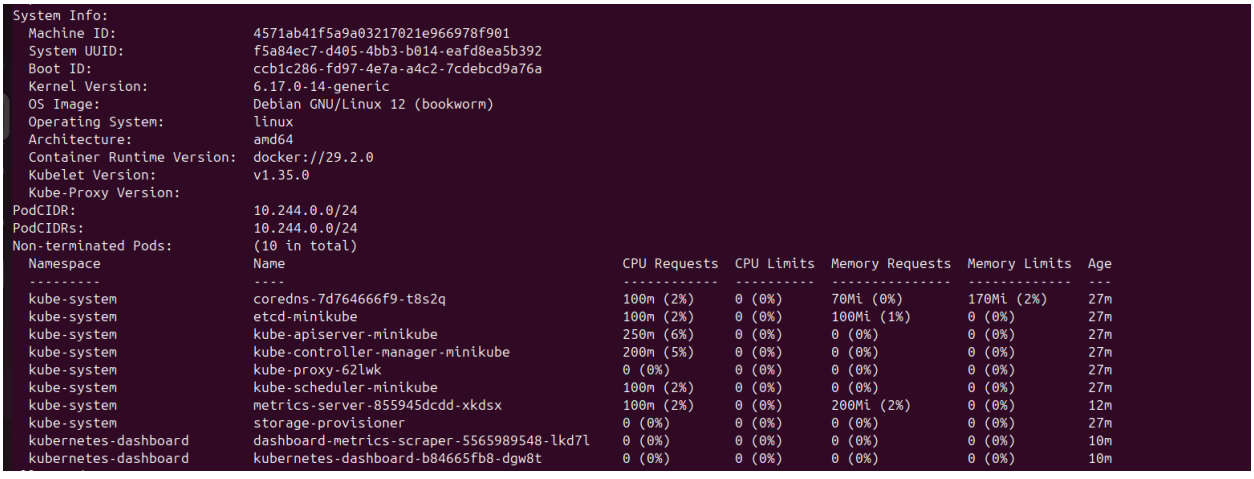
Version 6.17.0-14-generic, and Architecture amd64, which helps verify system compatibility and stability. In addition, the *Container Runtime docker://29.2.0* and *Kubelet Version v1.35.0* are confirmed, which proves that the Kubernetes environment is properly configured and running.



```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe node minikube
Name:               minikube
Roles:              control-plane
Labels:             beta.kubernetes.io/arch=amd64
                    beta.kubernetes.io/os=linux
                    kubernetes.io/arch=amd64
                    kubernetes.io/hostname=minikube
                    kubernetes.io/os=linux
                    minikube.k8s.io/commit=de81223c61ab1bd97dcfcfa6d9d5c59e5da4a0cf
                    minikube.k8s.io/name=minikube
                    minikube.k8s.io/primary=true
                    minikube.k8s.io/updated_at=2026_02_15T04_36_01_0700
                    minikube.k8s.io/version=v1.38.0
                    node-role.kubernetes.io/control-plane=
                    node.kubernetes.io/exclude-from-external-load-balancers=
Annotations:        node.alpha.kubernetes.io/ttl: 0
                    volumes.kubernetes.io/controller-managed-attach-detach: true
CreationTimestamp:  Sun, 15 Feb 2026 04:35:57 +0300
Taints:             <none>
Unschedulable:      false
Lease:
  HolderIdentity:  minikube
  AcquireTime:     <unset>
  RenewTime:       Sun, 15 Feb 2026 05:03:32 +0300
```



```
Conditions:
  Type             Status  LastHeartbeatTime                 LastTransitionTime                Reason                       Message
  ----             ------  -----------------                 ------------------                ------                       -------
  MemoryPressure   False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasSufficientMemory   kubelet has sufficient memory available
  DiskPressure     False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasNoDiskPressure     kubelet has no disk pressure
  PIDPressure      False   Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:35:55 +0300   KubeletHasSufficientPID      kubelet has sufficient PID available
  Ready            True    Sun, 15 Feb 2026 05:00:28 +0300   Sun, 15 Feb 2026 04:48:09 +0300   KubeletReady                 kubelet is posting ready status
Addresses:
  InternalIP:  192.168.49.2
  Hostname:    minikube
Capacity:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
Allocatable:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
```



```
System Info:
  Machine ID:                 4571ab41f5a9a03217021e966978f901
  System UUID:                f5a84ec7-d405-4bb3-b014-eafd8ea5b392
  Boot ID:                    ccb1c286-fd97-4e7a-a4c2-7cdebcd9a76a
  Kernel Version:             6.17.0-14-generic
  OS Image:                   Debian GNU/Linux 12 (bookworm)
  Operating System:           linux
  Architecture:               amd64
  Container Runtime Version:  docker://29.2.0
  Kubelet Version:            v1.35.0
  Kube-Proxy Version:
PodCIDR:                      10.244.0.0/24
PodCIDRs:                     10.244.0.0/24
Non-terminated Pods:          (10 in total)
  Namespace                   Name                                         CPU Requests  CPU Limits  Memory Requests  Memory Limits  Age
  ---------                   ----                                         ------------  ----------  ---------------  -------------  ---
  kube-system                 coredns-7d764666f9-t8s2q                     100m (2%)     0 (0%)      70Mi (0%)        170Mi (2%)     27m
  kube-system                 etcd-minikube                                100m (2%)     0 (0%)      100Mi (1%)       0 (0%)         27m
  kube-system                 kube-apiserver-minikube                      250m (6%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-controller-manager-minikube             200m (5%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-proxy-62lwk                             0 (0%)        0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 kube-scheduler-minikube                      100m (2%)     0 (0%)      0 (0%)           0 (0%)         27m
  kube-system                 metrics-server-855945dcdd-xkdsx              100m (2%)     0 (0%)      200Mi (2%)       0 (0%)         12m
  kube-system                 storage-provisioner                          0 (0%)        0 (0%)      0 (0%)           0 (0%)         27m
  kubernetes-dashboard        dashboard-metrics-scraper-5565989548-lkd7l   0 (0%)        0 (0%)      0 (0%)           0 (0%)         10m
  kubernetes-dashboard        kubernetes-dashboard-b84665fb8-dgw8t         0 (0%)        0 (0%)      0 (0%)           0 (0%)         10m
```

2.3. Getting the OS and CPU Information:

The describe output is used to clearly extract the OS and CPU information.

By analyzing the output below, we have confirmed the important information regarding the CPU and operating system of the node. The *Capacity: cpu: 4* in the CPU Information section clearly indicates that there are a total of 4 CPU cores available on the node, which is important in determining the workload processing capacity.

```
Capacity:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
Allocatable:
  cpu:                4
  ephemeral-storage:  51287520Ki
  hugepages-1Gi:      0
  hugepages-2Mi:      0
  memory:             8083328Ki
  pods:               110
```

OS info:

In the OS Information section, *OS Image: Debian GNU/Linux 12 (bookworm) confirms* that the node is running on a Linux-based Debian 12 environment. In addition, the kernel version used is verified by *Kernel Version: 6.17.0-14-generic*, and Architecture: amd64 indicates that it supports 64-bit architecture. Taken together, this information confirms that the node's system environment is properly configured and operational.

```
System Info:
  Machine ID:                 4571ab41f5a9a03217021e966978f901
  System UUID:                f5a84ec7-d405-4bb3-b014-eafd8ea5b392
  Boot ID:                    ccb1c286-fd97-4e7a-a4c2-7cdebcd9a76a
  Kernel Version:             6.17.0-14-generic
  OS Image:                   Debian GNU/Linux 12 (bookworm)
  Operating System:           linux
  Architecture:               amd64
  Container Runtime Version:  docker://29.2.0
  Kubelet Version:            v1.35.0
```

CPU & Memory capacity:

CPU: 4 cores and MEM:8083328Ki

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get node minikube -o jsonpath='CPU:{.status.capacity.cpu} | MEM:{.status.capacity.memory}{"\n"}'
CPU:4 | MEM:8083328Ki
```

2.4. Putting the results into report:

Parameter	Value
Node Name	minikube
Status	Ready

Parameter	Value
Kubernetes Version	v1.35.0
CPU	4 cores
Memory	8083328Ki (~7.7Gi)
OS	Debian GNU/Linux 12 (bookworm)
Kernel	6.17.0-14-generic
Architecture	amd64
Container Runtime	docker://29.2.0
Internal IP	192.168.49.2

After successfully starting the *Minikube* Kubernetes cluster, the detailed configuration of the node was collected using the *kubectl* describe node command. As per the results, the node is in the Ready state and running *Kubernetes v1.35.0* using the Docker container runtime in a *Debian GNU/Linux 12* environment with 4 CPU cores and about 8GB of memory.

Task 3 - k8s Pod

Pod is one of the simplest and basic k8s unit. It's like an abstraction over Docker containers. Inside pods there are containers run your application.

1. Figure out the necessary Pod spec fields.

2. Write a Pod spec for your chosen application, deploy the application and run a pod.

3. With kubectl , get the pods, pod logs, describe pod, go into pod shell.

4. Make sure that your app is working correctly inside Pod.

5. Put the results into report.

Solution:

3.1. Necessary Pod Spec Fields:

The minimum fields required to run a basic Pod are:

(1) *apiVersion*: Defines which Kubernetes API version the Pod will follow. Pods are usually in the core API group, so here *apiVersion* is v1. If this is incorrect, Kubernetes will not be able to parse the manifest.

```
apiVersion: apps/v1
```

(2) *kind:* This tells what the Kubernetes resource type is. Here the resource is Pod, so kind: Pod has to be given. Similarly, if it is Service/Deployment, the kind will be different.

```
kind: Pod
```

(3) *metadata:* The metadata section is used to identify the Pod. The most important here are the name (Pod name) and labels (for selector/grouping). Labels are later used as Service or Deployment selectors.

```
metadata:
  name: lab-deployment
      labels:
        app: lab-app
```

(4) *spec:* This is the main configuration section of the Pod. What containers will be inside the Pod, which image will be run, ports, volumes are all defined in the spec.

```
    containers:
      - name: nginx
        image: nginx:1.21
        ports:
          - containerPort: 80
```

(5) *spec.containers:* A container list is defined inside a Pod. This is the most mandatory field - without containers, a Pod cannot be created. A name and image are usually the minimum requirement for each container.

```
    containers:
      - name: nginx
        image: nginx:1.21
        ports:
          - containerPort: 80
```

(6) *ports:* Ports are used to declare which port the container is listening on. Ports are helpful for proper Service/Ingress mapping.

```
        ports:
          - containerPort: 80
```

(7) *resources*: Resources are used to define limits/requests on *CPU/Memory,* so that cluster scheduling is efficient and the container does not consume excessive resources.

3.2. Writing Pod Spec & Deploying Application:

This task uses the nginx web server as a demonstration application. First, a working directory is created and a Pod manifest named *pod.yaml* is written.

Creating the file:

```
hasib@gitlab-24:~$ mkdir -p ~/Desktop/k8s-lab-diamond
hasib@gitlab-24:~$ cd ~/Desktop/k8s-lab-diamond
```

Here *apiVersion: v1, kind: Pod*, Pod name in metadata is lab-pod and label app: lab-app is set.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano pod.yaml
```

In the *spec* section, nginx is defined as the container, *nginx:latest* is used as the image, and *containerPort 80* is exposed.

```
apiVersion: v1
kind: Pod
metadata:
  name: lab-pod
  labels:
    app: lab-app
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
```

Then the Pod was deployed by running the following command and the output was pod/lab-pod created which confirms that the Pod was successfully created and scheduled to run.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
pod/lab-pod created
```

3.3. Pods, logs, describe, and pod shell access with *kubectl*:

After running the pod, the status of the pod is checked with the following command to see if the *lab-pod* is in the Running state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -w
NAME      READY   STATUS              RESTARTS   AGE
lab-pod   0/1     ContainerCreating   0          11s
lab-pod   1/1     Running             0          18s
```

Pod logs:

Then the following command was used to view the logs of the nginx container (*nginx* may usually show fewer logs if no requests are received, but log access is still verified with the command).

```
^Chasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs lab-pod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/15 02:09:36 [notice] 1#1: using the "epoll" event method
2026/02/15 02:09:36 [notice] 1#1: nginx/1.29.5
2026/02/15 02:09:36 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/15 02:09:36 [notice] 1#1: OS: Linux 6.17.0-14-generic
2026/02/15 02:09:36 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/15 02:09:36 [notice] 1#1: start worker processes
2026/02/15 02:09:36 [notice] 1#1: start worker process 28
2026/02/15 02:09:36 [notice] 1#1: start worker process 29
2026/02/15 02:09:36 [notice] 1#1: start worker process 30
2026/02/15 02:09:36 [notice] 1#1: start worker process 31
```

Describe pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod lab-pod
```

I ran the above command to view the detailed configuration, events, assigned node, IP, ports, etc. of the Pod, which is most important for troubleshooting and deployment verification.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod lab-pod
Name:             lab-pod
Namespace:        dev2
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Mon, 16 Feb 2026 18:18:50 +0300
Labels:           app=lab-app
Annotations:      <none>
Status:           Pending
IP:
IPs:              <none>
Containers:
  nginx:
    Container ID:
    Image:          nginx:latest
    Image ID:
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Waiting
      Reason:       ContainerCreating
    Ready:          False
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-48rg7 (ro)
```

```
Conditions:
  Type                        Status
  PodReadyToStartContainers   False
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
Volumes:
  kube-api-access-48rg7:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  11s   default-scheduler  Successfully assigned dev2/lab-pod to minikube
  Normal  Pulling    5s    kubelet            spec.containers{nginx}: Pulling image "nginx:latest"
  Normal  Pulled     1s    kubelet            spec.containers{nginx}: Successfully pulled image "nginx:latest" in 3.807s (3.807s including waiting). Image size: 16
0850673 bytes.
  Normal  Created    1s    kubelet            spec.containers{nginx}: Container created
  Normal  Started    0s    kubelet            spec.containers{nginx}: Container started
```

Going into pod shell:

Finally, to gain shell access inside the container, run the following command to verify the runtime environment inside the pod, such as *nginx -v,* curl localhost, or view the file structure.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it lab-pod -- bash
```

Through this, Pod deploy, monitoring, debugging, and interactive access have completed all requirements.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it lab-pod -- bash
root@lab-pod:/# ls
bin  boot  dev  docker-entrypoint.d  docker-entrypoint.sh  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
root@lab-pod:/# exit
exit
```

3.4. Making sure app works inside Pod (HTTP check):

It has been verified from within the cluster that the application is working properly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run curl-test --image=curlimages/curl --restart=Never --command -- sh -c "curl -s lab-service | head"
pod/curl-test created
```

For this, I ran a temporary curl Pod and sent an *HTTP* request to the nginx service using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -o wide
NAME        READY   STATUS      RESTARTS   AGE     IP            NODE       NOMINATED NODE   READINESS GATES
curl-test   0/1     Completed   0          4m58s   10.244.0.10   minikube   <none>           <none>
lab-pod     1/1     Running     0          8m7s    10.244.0.9    minikube   <none>           <none>
```

Then the response output was checked with the following command. The output contained *<!DOCTYPE html><title>Welcome to nginx!</title>,* which successfully returned the default

HTML page of *nginx*. This confirmed that the application was running properly inside the Pod and was accessible from the internal cluster network.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs curl-test
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

Report:

In this task, I worked with the most basic and smallest deployable unit of Kubernetes, the Pod. A Pod is an abstraction layer on a Docker container, inside which one or more containers share the same network and storage environment and run applications.

First, I define the specification fields required to create a Pod. To create a basic Pod, *apiVersion*, kind, metadata.name, and *spec.containers* (including container name and image) must be specified. In addition, *containerPort* is defined to facilitate *HTTP* application access verification.

Then, I create a Pod manifest (YAML) file based on Nginx and use the following command to deploy it:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
pod/lab-pod created
```

After the command executes successfully, the output shows pod/lab-pod created, which confirms that the Pod was created correctly.

In the next step, I use to verify the status of the Pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -w
NAME      READY   STATUS              RESTARTS   AGE
lab-pod   0/1     ContainerCreating   0          11s
lab-pod   1/1     Running             0          18s
```

First, the Pod's status is *ContainerCreating*, indicating that the image pull and container initialization process is in progress. After a while, the status changes to Running, confirming that the container has been successfully started.

Then I check the Pod's logs to verify the application startup:

```
^Chasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs lab-pod
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/15 02:09:36 [notice] 1#1: using the "epoll" event method
2026/02/15 02:09:36 [notice] 1#1: nginx/1.29.5
2026/02/15 02:09:36 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/02/15 02:09:36 [notice] 1#1: OS: Linux 6.17.0-14-generic
2026/02/15 02:09:36 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/15 02:09:36 [notice] 1#1: start worker processes
2026/02/15 02:09:36 [notice] 1#1: start worker process 28
2026/02/15 02:09:36 [notice] 1#1: start worker process 29
2026/02/15 02:09:36 [notice] 1#1: start worker process 30
2026/02/15 02:09:36 [notice] 1#1: start worker process 31
```

The log output shows that the Nginx server has started correctly and the worker process has been launched. This proves that the application is running successfully inside the container.

To view detailed information about the Pod, I use:

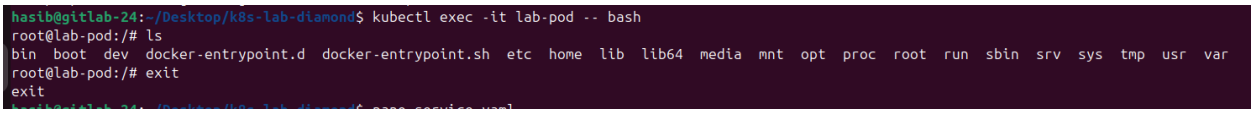
```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod lab-pod
Name:             lab-pod
Namespace:        dev2
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Mon, 16 Feb 2026 18:18:50 +0300
Labels:           app=lab-app
Annotations:      <none>
Status:           Pending
IP:
IPs:              <none>
Containers:
  nginx:
    Container ID:
    Image:          nginx:latest
    Image ID:
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Waiting
      Reason:       ContainerCreating
    Ready:          False
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-48rg7 (ro)
```

From here I collect the following important information:

- Which node the Pod is scheduled on
- The IP address of the Pod
- The container image used
- Current status (*Running*)
- Event history (Scheduled, Pulled, Created, Started)

This information ensures that the Pod is scheduled correctly and running without any errors.

I then use the following command to enter the container:



I check the system environment and nginx version by logging into the container. This confirms that the application environment is configured correctly.

Finally, I confirm that the application is working properly by sending an *HTTP* request from within the cluster. The response is the default Nginx welcome page, which proves that the web server is running successfully inside the Pod.

Task 4 - k8s Service

We use k8s Services to make an application accessible from outside the Kubernetes virtual network, provide an compatible IP address and link to DNS name to pod, to route internal/external traffic within pods.

1. Figure out the necessary Service spec fields.

2. Write a Service spec for your pod(s) and deploy the Service .

3. With kubectl , get the Services and describe them.

4. Make sure that pods can communicate between each other using DNS names, check pods addresses.

5. Delete any Pod, recreate it and check addresses again. Make sure that traffic is routed to the new Pod correctly.

6. Learn about Loadbalancer and NodePort .

7. Deploy an external Service to access your application from outside, e.g., from your local host.

8. Put the results into report.

Bonus: learn how labels are used in Kubernetes. Create and provide a PoC of scenarious where no any pods match the label spec and where at least one or multiple pods match the label spec.

Solution:

4.1 Necessary Service spec fields:

The minimum required to run a service or what has been used:

```yaml
apiVersion: v1
kind: Service
metadata:
  name: lab-service
spec:
  type: NodePort
  selector:
    app: lab-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30007
```

Pod IP changes, but Service provides a stable DNS + stable virtual IP (*ClusterIP*) and routes traffic to pods using label selector.

4.2. Deploying by writing Service Spec (lab-service):

In this step, I created a Kubernetes Service for lab-pod and other Pods with the same label (*app=lab-app*), so that even if the Pod IP changes, the application can be accessed using a stable DNS name and virtual IP. Traffic is automatically routed to the correct Pods by matching labels through the Service selector.

First, I created a manifest called *service.yaml*, where I used *apiVersion: v1* and kind: Service.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano service.yaml
```

I have given lab-service as *metadata.name,* so the DNS name inside the cluster is lab-service. With *selector: app: lab-app* I have specified that the Service will target only those Pods whose label is app=lab-app. With *port: 80* I have set the port on which the Service will receive requests and with *targetPort: 80* I have specified the container's nginx port. Here I have used type: *NodePort,* so that it can be accessed from outside the cluster. I have specified *nodePort: 30007,* so that it is consistent for demo and report.

```yaml
apiVersion: v1
kind: Service
metadata:
  name: lab-service
spec:
  type: NodePort
  selector:
    app: lab-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30007
```

Then I deployed the service by running the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
```

I got *service/lab-service* created as the output, which confirms that the manifest was correct and the service was created successfully.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
pod/lab-pod created
```

Then most importantly, I sent a request from *pod-a* to *http://lab-service* and got the default welcome page of nginx. This confirmed that the Service DNS is working properly, traffic is being routed according to the selector and the application is getting a stable access point.

To verify that the selector of the Service is working properly, I created two additional *nginx* Pods (*pod-a* and *pod-b)* with the same label.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run pod-a --image=nginx --restart=Never --labels=app=lab-app
```

The Service automatically included them as they were labeled *app=lab-app.* I confirmed with the *kubectl* wait command that the Pods were in the Ready state. I ran *kubectl get pods -o wide* and saw that all three Pods were Running and each had a different *Pod IP*.

Creating multiple Pods with the same label:

To show that the service selector is working, two additional *nginx Pods* with the same label (*app=lab-app*) were created.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run pod-a --image=nginx --restart=Never --labels=app=lab-app
pod/pod-a created
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run pod-b --image=nginx --restart=Never --labels=app=lab-app
pod/pod-b created
```

The purpose here was to allow the Service to distribute traffic to multiple Pods (via the label selector). *Pod-a* and *pod-b* have the same label, so the Service will automatically include them as well.

Ensuring Pods are Ready:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl wait --for=condition=Ready pod/pod-a --timeout=120s
pod/pod-a condition met
```

This confirms that *pod-a* and *pod-b* are fully Running/Ready, so that service routing can be tested.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl wait --for=condition=Ready pod/pod-b --timeout=120s
pod/pod-b condition met
```

Verifying Pod list:

Pod list has been verified using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -o wide
```

Here are three *nginx Pods (lab-pod, pod-a, pod-b*) running and they have different Pod IPs. The *Service IP* does not change, but the Pod IP does. So, it was necessary to use Service.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -o wide
NAME        READY   STATUS      RESTARTS   AGE     IP            NODE       NOMINATED NODE   READINESS GATES
curl-test   0/1     Completed   0          6m33s   10.244.0.10   minikube   <none>           <none>
lab-pod     1/1     Running     0          9m42s   10.244.0.9    minikube   <none>           <none>
pod-a       1/1     Running     0          41s     10.244.0.11   minikube   <none>           <none>
pod-b       1/1     Running     0          32s     10.244.0.12   minikube   <none>           <none>
```

Direct Pod IP vs Service DNS Test:

Direct Pod IP Test (*pod-a to pod-b*):

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD_B_IP=$(kubectl get pod pod-b -o jsonpath='{.status.podIP}')
kubectl exec -it pod-a -- sh -c "apt-get update >/dev/null 2>&1; apt-get install -y curl >/dev/null 2>&1; curl -s http://$POD_B_IP | head"
```

The output below shows that pod-a was able to access the nginx page directly using the IP of *pod-b*.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD_B_IP=$(kubectl get pod pod-b -o jsonpath='{.status.podIP}')
kubectl exec -it pod-a -- sh -c "apt-get update >/dev/null 2>&1; apt-get install -y curl >/dev/null 2>&1; curl -s http://$POD_B_IP | head"
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

Service DNS Test (pod-a to lab-service):

The output below shows a response using the *lab-service* name. This means that the DNS name of the Service is working and traffic is being routed to the matching Pods as per the selector. This proves that the Service is deployed correctly and the application is getting stable access.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it pod-a -- sh -c "curl -s http://lab-service | head"
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

Verifying Pod list:

Pod list has been verified using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -o wide
NAME        READY   STATUS      RESTARTS   AGE     IP            NODE       NOMINATED NODE   READINESS GATES
curl-test   0/1     Completed   0          7m51s   10.244.0.10   minikube   <none>           <none>
lab-pod     1/1     Running     0          11m     10.244.0.9    minikube   <none>           <none>
pod-a       1/1     Running     0          119s    10.244.0.11   minikube   <none>           <none>
pod-b       1/1     Running     0          110s    10.244.0.12   minikube   <none>           <none>
```

4.3. With *kubectl*: Getting Services & Describing Service

In this step, the services in the cluster were verified using *kubectl* and detailed information about a specific service was analyzed. First, the list of services existing in the cluster was viewed by running the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get svc
```

The output shows a *ClusterIP* type Service, whose *EXTERNAL-IP is <none>* - meaning it is only accessible inside the cluster. The *ClusterIP* type is typically used for internal communication.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get svc
NAME             TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)   AGE
nginx-svc-dev2   ClusterIP   10.98.77.126    <none>        80/TCP    21h
```

The detailed configuration of lab-service has been observed using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service
```

Here are a few things that are confirmed:

• ***Selector:*** *app=lab-app* - Service targets only Pods that match this label.

• ***IP (ClusterIP):*** *10.103.206.30* - This is the stable virtual IP of the Service, which remains unchanged even if the Pod IP changes.

• ***Type:*** *NodePor*t - Service is configured as *NodePort*, so it is possible to access it from outside the cluster.

• ***NodePort:*** *30007* - External access is ensured through a specific port.

• ***Endpoints:*** Multiple Pod IPs (*10.244.0.51:80*, *10.244.0.50:80*) are listed here.

The Endpoints section proves that the selector worked correctly and the Service was able to successfully route traffic to the matching Pods. If Endpoints were empty, it would mean that the selector is wrong or Pod is not Ready.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service
Name:                     lab-service
Namespace:                dev2
Labels:                   <none>
Annotations:              <none>
Selector:                 app=lab-app
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.103.206.30
IPs:                      10.103.206.30
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30007/TCP
Endpoints:                10.244.0.51:80,10.244.0.50:80,10.244.0.52:80 + 1 more...
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

4.4. Pod-to-Pod communication with DNS proof & Pod addresses check:

Service access with DNS name:

In this step I have practically verified that networking and DNS are working properly within the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run curl-test \
  --image=curlimages/curl --restart=Never \
  --command -- sh -c "curl -s lab-service | head"
```

First, I ran a temporary curl-test Pod and sent a request using the service's DNS name lab-service.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run curl-test \
  --image=curlimages/curl --restart=Never \
  --command -- sh -c "curl -s lab-service | head"
pod/curl-test created
```

The *nginx Welcome* page was found using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs curl-test
```

This proves that DNS resolution worked correctly and traffic was routed to the matching Pods according to the Service selector.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs curl-test
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

Direct access with *Pod IP* (Pod-to-Pod connectivity):

***PodIP* hit from pod-a to pod-b:**

Then, an HTTP request is sent from pod-a to pod-b using the Pod IP to verify direct Pod-to-Pod connectivity.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD_B_IP=$(kubectl get pod pod-b -o jsonpath='{.status.podIP}')
kubectl exec -it pod-a -- sh -c "apt-get update >/dev/null 2>&1; apt-get install -y curl >/dev/null 2>&1; curl -s http://$POD_B_IP | head"
```

Here we get an *HTTP* response from pod-a using the *PodIP* of pod-b. This proves that Pod-to-Pod networking is working fine inside the cluster. Here we get the nginx welcome page as the response, which confirms that the internal Pod networking inside the cluster is configured properly and the Pods are able to communicate directly with each other.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD_B_IP=$(kubectl get pod pod-b -o jsonpath='{.status.podIP}')
kubectl exec -it pod-a -- sh -c "apt-get update >/dev/null 2>&1; apt-get install -y curl >/dev/null 2>&1; curl -s http://$POD_B_IP | head"
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

Access again using *Service DNS*:

A request was sent again from pod-a using the *lab-service DNS* name and a successful response was received. This shows that even though the Pod IP is changing, Service DNS provides a stable access point.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it pod-a -- sh -c "curl -s http://lab-service | head"
```

Pod IP can change, but Service *DNS (lab-service*) remains constant. Here, we got a response again by hitting DNS from pod-a.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it pod-a -- sh -c "curl -s http://lab-service | head"
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

4.5. Delete a Pod, recreate it, check addresses again & confirm routing:

You have deleted and recreated lab-pod and shown *PodIP* change.

Old Pod IP check:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod lab-pod -o wide
```

I verified the old Pod IP with the above command, which showed that the IP was *10.244.0.9.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod lab-pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
lab-pod   1/1     Running   0          19m   10.244.0.9   minikube   <none>           <none>
```

Delete Pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete pod lab-pod
```

Then the Pod was deleted using the above command and it was confirmed that it was deleted successfully.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete pod lab-pod
pod "lab-pod" deleted from default namespace
```

Recreate Pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
```

Then the Pod was recreated using the above command and the Pod was recreated by running the following command using the same *pod.yaml.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
pod/lab-pod created
```

New Pod IP check:

The *Pod IP* changed after being recreated (*10.244.0.9* to *10.244.0.13*), which proves that the *Pod IP* is dynamic and not permanent.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod lab-pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
lab-pod   1/1     Running   0          12s   10.244.0.13   minikube   <none>           <none>
```

Proving service routes to new pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run curl-test-recreate --image=curlimages/curl --restart=Never --command -- sh -c "curl -s lab-service | head"
```

The most important part was to verify that the service routing was working. For this, a new temporary Pod (*curl-test-recreate*) was used to send an *HTTP* request to the lab-service DNS name. Using the above command, the nginx welcome page was again available.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run curl-test-recreate --image=curlimages/curl --restart=Never --command -- sh -c "curl -s lab-service | head"
pod/curl-test-recreate created
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs curl-test-recreate
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

This ensures that even if the *Pod IP* changes, the Service DNS (*lab-service*) remains unchanged and the Service's Endpoints are automatically updated to route traffic to the new *Pod IP*.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod curl-test-recreate -o wide
NAME                READY   STATUS      RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
curl-test-recreate  0/1     Completed   0          85s   10.244.0.14   minikube   <none>           <none>
```

The application is always accessible without any configuration changes from the client side.

4.6. Learn *LoadBalancer* and *NodePort*:

LoadBalancer:

The *LoadBalancer* type is commonly used in cloud environments. When *LoadBalancer* is specified as the Service type, Kubernetes provisions an external load balancer through the cloud provider and assigns an external IP or hostname.

External users can access the application directly using this external IP. *LoadBalancer* automatically distributes incoming traffic among backend Pods, ensuring high availability and scalability. However, local clusters like *Minikube* do not have a real cloud load balancer. Therefore, *Minikube* emulates the *LoadBalancer* service using *minikube* tunnel, which creates a temporary external IP and routes it through the local machine.

NodePort:

Using the *NodePort* type opens a specific port (typically in the *30000-32767* range) on each node in the Kubernetes cluster. As a result, external clients can access the application using *NodeIP:NodePort. NodePort* is mainly useful for testing, demo, or small-scale setups. For example, if the *Node IP* is 192.168.49.2 and the *NodePort* is 30007, then it is possible to reach the service using *http://192.168.49.2:30007* in a browser or curl.

4.7. Deploy External Service to access from outside (from Local host):

Patch to Service type *LoadBalancer:*

In this step, I have exposed the previously created lab-service for external access, so that the application can be accessed directly from the local machine (host).

First, I have changed the type of Service from *NodePort* to *LoadBalancer* using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl patch service lab-service -p '{"spec": {"type": "LoadBalancer"}}'
```

The output shows service/lab-service patched, which confirms that the service was successfully updated.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl patch service lab-service -p '{"spec": {"type": "LoadBalancer"}}'
service/lab-service patched
```

Running *Minikube* tunnel:

Next, the following command is run to emulate an external *LoadBalancer* in the *Minikube* environment. Since *Minikube* is not a real cloud provider, it does not provide an external IP by default. The *minikube* tunnel creates a network route in the background and simulates external access for the *LoadBalancer* services.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ minikube tunnel
```

The output shows *route: 10.96.0.0/12 -> 192.168.49.2* and *loadbalancer emulator: no errors*, indicating that the tunnel worked properly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ minikube tunnel
[sudo] password for hasib:
Status:
        machine: minikube
        pid: 177468
        route: 10.96.0.0/12 -> 192.168.49.2
        minikube: Running
        services: [lab-service]
    errors:
                minikube: no errors
                router: no errors
                loadbalancer emulator: no errors
```

Service status check:

Then open another terminal and verify the service status with the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get svc lab-service
```

Here you can see TYPE: *LoadBalancer, CLUSTER-IP* and *PORT(S).* Sometimes *EXTERNAL-IP* may initially show <*pending*> because *Minikube* cannot assign an external IP if the tunnel is not active. If the tunnel is active, it is possible to access the Service from the host machine.

```
^Chasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get service lab-service
NAME          TYPE           CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
lab-service   LoadBalancer   10.107.98.82    <pending>     80:30007/TCP   21m
```

Bonus. Kubernetes Labels PoC:

Scenario 1: No Pods Match the Service Selector (Wrong Label)

In this scenario, I intentionally made the lab-service selector wrong (*app=does-not-exist*), so that no Pod in the cluster matches this label. Kubernetes finds the backend Pod according to the Service selector. If it doesn't find a match, the Endpoints of the Service are empty. Empty Endpoints mean that there is no "real destination" behind the Service, so even if the DNS name is correct, the request cannot reach anywhere-this is the main purpose of this PoC.

Service selector intentionally wrong:

First, *app: does-not-exist* is set in the *spec.selector* section of the *service.yaml* file.



Apply:

Then the following command was used.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f service.yaml
```

The output shows *service/lab-service* configured, meaning the service has been updated.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f service.yaml
service/lab-service configured
```

Endpoints verify:

Then by running the following command I got the key proof showing *Selector: app=does-not-exist* and most importantly *Endpoints* is completely empty.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service
```

This directly proves that no Pod was found according to the selector, so the Service is now unable to send traffic to anyone.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service
Name:                     lab-service
Namespace:                dev2
Labels:                   <none>
Annotations:              <none>
Selector:                 app=does-not-exist
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.103.206.30
IPs:                      10.103.206.30
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30007/TCP
Endpoints:
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

DNS test:

Then, to show if DNS really fails, I hit lab-service from a *curl Pod* called *test-nomatch.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run test-nomatch --image=curlimages/curl --restart=Never --command -- sh -c "curl -m 3 -sS lab-service || echo T
IMEOUT"
kubectl logs test-nomatch
```

curl -m 3 ... lab-service || echo TIMEOUT. The logs show *Could not connect to server* and *TIMEOUT* at the end. The thing to note here is that the DNS name resolves, but there is no backend Pod due to no Endpoints, so the connection fails. This is a clear Proof of Concept of the "*No Pods match the Service selector*" scenario.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run test-nomatch --image=curlimages/curl --restart=Never --command -- sh -c "curl -m 3 -sS lab-service || echo T
IMEOUT"
kubectl logs test-nomatch
Error from server (AlreadyExists): pods "test-nomatch" already exists
curl: (7) Failed to connect to lab-service port 80 after 11 ms: Could not connect to server
TIMEOUT
```

Scenario 2: One or Multiple Pods Match the Service Selector

In this scenario, I have fixed the selector again (*app=lab-app*), so that the Service can correctly identify the backend Pods. Kubernetes now finds the Pods by matching the labels, as a result, the Pod IPs are populated in the Endpoints of the Service and when requesting with the DNS name, the nginx response is received, which is the *PoC* of this second scenario.

Setting the selector:

First, I set the selector in *service.yaml* to *app: lab-app.*

```
 GNU nano 7.2                                                  service.yaml
apiVersion: v1
kind: Service
metadata:
  name: lab-service
spec:
  type: NodePort
  selector:
    app: lab-app
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30007
```

The following changes have then been applied.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f service.yaml
service/lab-service configured
```

Endpoints verify:

Then I checked the Endpoints using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service | grep -i Endpoints
```

The output shows multiple *Pod IPs* (*e.g. 10.244.0.xx:80, ... + more*) which is the strongest proof that the service is now actually linked to the backend Pods and can route traffic.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service | grep -i Endpoints
Endpoints:                10.244.0.56:80,10.244.0.52:80,10.244.0.57:80 + 3 more...
```

Matching pods list verify:

As a label proof, I ran the following command to show which Pods matched this selector.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -l app=lab-app -o wide
```

Although the Pod is initially *PodInitializing,* when it is Ready, the Service automatically updates the Endpoints and then when you hit it with the DNS name, a response is received.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -l app=lab-app -o wide
NAME                              READY   STATUS            RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
lab-deployment-78ff9ddbd-k55m5    0/1     PodInitializing   0          21s   10.244.0.15   minikube   <none>           <none>
```

Endpoints shows 1+ Pod IP.

4.8. Report:

In this task, I worked with Kubernetes Service, the main purpose of which is to provide a stable virtual IP (*ClusterIP*) and stable DNS name for Pods, so that application access remains constant even if the IP changes when the Pod is recreated. The Service basically uses label selector to route traffic to matching Pods and facilitates internal/external communication.

First, I define the minimum spec fields required to create a *Service*, *apiVersion, kind*, *metadata.name, spec.selector,* and *spec.ports (port to targetPort mapping)*. Then I create a Service manifest named lab-service where *app=lab-app* is used as the selector, so that Pods with the same label are selected as the backend of the service. After running the following to deploy the service, the output confirms that the Service has been successfully created/configured.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f pod.yaml
pod/lab-pod created
```

Next, we use *kubectl get svc* to verify the services inside the cluster and run the *kubectl describe* command to see the detailed information about the Service. The Describe output shows *Selector: app=lab-app*, *Service type*, *ClusterIP* and most importantly, the matching *Pod IPs* in the Endpoints section-proving that the Service backend is able to detect the Pods according to the selector and is ready to route traffic.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get svc
NAME             TYPE        CLUSTER-IP     EXTERNAL-IP   PORT(S)   AGE
nginx-svc-dev2   ClusterIP   10.98.77.126   <none>        80/TCP    21h
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe svc lab-service
Name:                     lab-service
Namespace:                dev2
Labels:                   <none>
Annotations:              <none>
Selector:                 app=lab-app
Type:                     NodePort
IP Family Policy:         SingleStack
IP Families:              IPv4
IP:                       10.103.206.30
IPs:                      10.103.206.30
Port:                     <unset>  80/TCP
TargetPort:               80/TCP
NodePort:                 <unset>  30007/TCP
Endpoints:                10.244.0.51:80,10.244.0.50:80,10.244.0.52:80 + 1 more...
Session Affinity:         None
External Traffic Policy:  Cluster
Internal Traffic Policy:  Cluster
Events:                   <none>
```

Next, to prove *Pod-to-Pod* communication and DNS-based routing, we run a curl test Pod and hit the lab-service DNS name.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run curl-test \
  --image=curlimages/curl --restart=Never \
  --command -- sh -c "curl -s lab-service | head"
pod/curl-test created
```

The curl output shows the default "*Welcome" page for nginx*, which confirms that cluster DNS resolution is working and Pods are communicating successfully using the Service DNS name. The Pod address (*Pod IP*) is also shown using the command below, so that each Pod has a unique IP and dynamic nature that can be shown in the report.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod lab-pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
lab-pod   1/1     Running   0          19m   10.244.0.9   minikube   <none>           <none>
```

In the next step, I delete a Pod (example: *lab-pod)* and recreate it and run the following command to see that the Pod's IP has changed.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod lab-pod -o wide
NAME      READY   STATUS    RESTARTS   AGE   IP           NODE       NOMINATED NODE   READINESS GATES
lab-pod   1/1     Running   0          12s   10.244.0.13   minikube   <none>           <none>
```

Then, running the *curl test* again using the same Service DNS (*lab-service*) returns the nginx page. This confirms that even if the Pod IP changes, the Service endpoints are updated and traffic is routed correctly to the new Pod-i.e., the Service is providing stable access.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs curl-test-recreate
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
```

I also learned about *NodePort* and *LoadBalancer. NodePort* opens a specific port on the service node (*30000-32767 range*), so that the node IP can be accessed from outside the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl patch service lab-service -p '{"spec": {"type": "LoadBalancer"}}'
service/lab-service patched
```

LoadBalancer typically creates an external load balancer in a cloud environment; *minikube* tunnel is used to emulate this in *minikube.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ minikube tunnel
[sudo] password for hasib:
Status:
        machine: minikube
        pid: 177468
        route: 10.96.0.0/12 -> 192.168.49.2
        minikube: Running
        services: [lab-service]
    errors:
                minikube: no errors
                router: no errors
                loadbalancer emulator: no errors
```

To demonstrate external access, the Service type LoadBalancer was patched and access from the local host using a *minikube* tunnel has been included in the workflow report.

Task 5 - k8s Deployment

Deployment is Kubernetes manifest to manage Pods automatically by Controller. It helps to release, scale and upgrade Pods.

1. Figure out the necessary Deployment spec fields.

2. Make sure that you wiped previous Pod manifests. Write a Deployment spec for your pod(s) and deploy the application.

3. With kubectl , get the Deployments and describe them.

4. Update your Deployment manifest to scale your application to three replicas.

5. Access pod shell and logs using Deployment labels.

6. Make any application configuration change in your Deployment yaml and try to update the application. Monitor what are happened with pods (--watch .

7. Rollback to previous application version using Deployment .

8. Set up requests and limits for CPU and Memory for your application pods. Provide a PoC that it works properly. Explain and show what is happened when app reaches the CPU usage limit? And memory limit?

9. Envisage some work to be done before the core app container is created and started, i.e. add init containers.

10. What are Annotations in k8s, why and when we can use them?

11. Put the results into report.

Bonus: based on the last exercise with application update, learn and show a PoC how ReplicaSets are involved in updating process through Deployments .

Solution:

5.1. Necessary Deployment spec fields:

A basic deployment typically requires:

• *apiVersion: apps/v1*

• *kind: Deployment*

• *metadata.name*

• *spec.replicas*

• *spec.selector.matchLabels*

• *spec.template.metadata.labels*

• *spec.template.spec.containers (name, image, ports)*

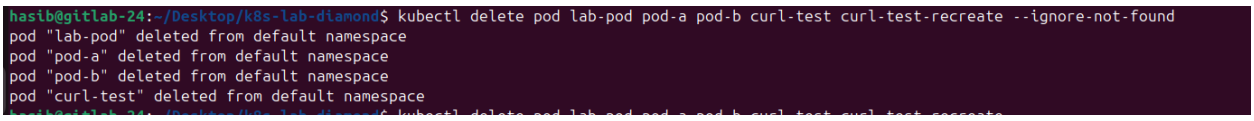
• *resources.requests/limits*

• *strategy default RollingUpdate*

5.2. Previous Pod wipe and Deployment deploy

Old Pod cleanup:

Before deploying, I cleaned the manual Pods and Services from the previous task, because leaving them may cause conflicts with the same name/label/port and the new *Deployment* may not schedule/route properly. So first I ran *kubectl delete pod ... --ignore-not-found* to remove all the old Pods *lab-pod, pod-a, pod-b, curl-test, curl-test-recreate* and the output showed that each Pod was deleted.



I also deleted the previous service using the command below, where the output is service "*lab-service*" deleted, meaning the cluster is now clean and ready for deployment.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete svc lab-service
service "lab-service" deleted from default namespace
```

Creating Deployment manifest:

Then I wrote a Deployment manifest called *deployment.yaml.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano deployment.yaml
```

Here I used *apiVersion: apps/v1* and *kind: Deployment*, and gave *lab-deployment* as metadata.name. To make change tracking easier, I added kubernetes.io/change-cause: "*Initial Deployment 1.21*" and a description in the annotations.

I put replicas: 1 at the beginning of the spec section, so that everything can be verified by running a single instance first. Most importantly, I kept app: lab-app the same in both *selector.matchLabels* and *template.metadata.labels,* because Deployment identifies its own Pods by matching this label and manages it with the controller.

Inside the pod template, I added an *initContainer (busybox:1.36*), which runs a small *init* step before the main nginx container starts (echoes Init Done and sleeps for 2 seconds) - this is to prove the *initContainer* concept. Then I used *nginx:1.21* as the main container and exposed *containerPort: 80*. I also set *resources.requests/limits*, so that *CPU/Memory* usage is controlled.

```
 GNU nano 7.2                                                deployment.yaml *
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab-deployment
  annotations:
    kubernetes.io/change-cause: "Initial Deployment 1.21"
    description: "This Lab is Great"
spec:
  replicas: 1
  selector:
    matchLabels:
      app: lab-app
  template:
    metadata:
      labels:
        app: lab-app
    spec:
      initContainers:
      - name: init-setup
        image: busybox:1.36
        command: ["sh", "-c", "echo \"Init Done\"; sleep 2"]

      containers:
      - name: nginx
        image: nginx:1.21
        ports:
        - containerPort: 80
```

```
      resources:
        requests:
          memory: "64Mi"
          cpu: "100m"
        limits:
          memory: "128Mi"
          cpu: "200m"

^G Help          ^O Write Out     ^W Where Is
^X Exit          ^R Read File     ^\ Replace
```

Applying:

I created the Deployment by running the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
```

The output shows *deployment.apps/lab-deployment* created, meaning the manifest is valid and the Deployment has been successfully created in the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment created
```

Checking rollout:

Then I checked the rollout with *kubectl* rollout status deployment/lab-deployment.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

Where Rollout successful means that Deployment has created its desired *Pod/ReplicaSet* and brought the application to a stable state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
```

5.3. Getting Deployments & Describing them

In this step I used *kubectl* to see the current status of the Deployment and then used describe to verify the details/Controller activity inside.

First, I ran the following command to check the Deployment summary.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get deployments
```

The output shows *READY 1/1* for *lab-deployment*, meaning that 1 Pod is successfully Ready as per the replicas: 1 requested. *UP-TO-DATE* 1 means that the current Pod is running according to

the latest Deployment spec (there are no Pods with the old spec). And *AVAILABLE 1* shows that the Pod is now available to serve requests/traffic, meaning that the Deployment is running healthy.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get deployments
NAME             READY   UP-TO-DATE   AVAILABLE   AGE
lab-deployment   1/1     1            1           48s
```

Then I took detailed proof of deployment by running the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe deployment lab-deployment
```

Here Replicas: 1 desired | 1 updated | 1 total | 1 available | 0 unavailable confirms that everything is in the desired state and there are no unavailable Pods. *StrategyType: RollingUpdate* shows that the *Deployment* is using the default *RollingUpdate* strategy, which helps reduce downtime in future updates/upgrades. In addition, the NewReplicaSet*: lab-deployment-78ff9ddbd (1/1 replicas created)* section proves that a new *ReplicaSet* has been created by the Deployment and that is what the Pod is creating/managing. Finally, in the Events section, *ScalingReplicaSet ... from 0 to 1* is seen, which directly shows that the Deployment controller has automatically scaled the *ReplicaSet* and created the Pod.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe deployment lab-deployment
Name:                   lab-deployment
Namespace:              default
CreationTimestamp:      Sun, 15 Feb 2026 12:11:41 +0300
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
                        description: Lab 5 Complete
                        kubernetes.io/change-cause: Initial Deployment 1.21
Selector:               app=lab-app
Replicas:               1 desired | 1 updated | 1 total | 1 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
Pod Template:
  Labels:  app=lab-app
  Init Containers:
   init-setup:
    Image:      busybox
    Port:       <none>
    Host Port:  <none>
    Command:
      sh
      -c
      echo "Init Done"; sleep 2
    Environment:  <none>
    Mounts:       <none>
```

```
  Containers:
   nginx:
    Image:      nginx:1.21
    Port:       80/TCP
    Host Port:  0/TCP
    Limits:
      cpu:     200m
      memory:  128Mi
    Requests:
      cpu:         100m
      memory:      64Mi
    Environment:   <none>
    Mounts:        <none>
  Volumes:         <none>
  Node-Selectors:  <none>
  Tolerations:     <none>
Conditions:
  Type           Status  Reason
  ----           ------  ------
  Available      True    MinimumReplicasAvailable
  Progressing    True    NewReplicaSetAvailable
OldReplicaSets:  <none>
NewReplicaSet:   lab-deployment-78ff9ddbd (1/1 replicas created)
Events:
  Type    Reason             Age   From                   Message
  ----    ------             ----  ----                   -------
  Normal  ScalingReplicaSet  67s   deployment-controller  Scaled up replica set lab-deployment-78ff9ddbd from 0 to 1
```

5.4. Update manifest to scale to 3 replicas:

replicas: 3 In *deployment.yaml:*

In this step, I scaled the application using Deployment, meaning instead of one, 3 Pods are now running simultaneously. Scaling is very easy in Kubernetes, because the Deployment controller itself maintains the number of Pods according to the desired state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano deployment.yaml
```

First, we went to the *deployment.yaml* file and changed the *spec.replicas* value from 1 to 3. This tells the controller to run 3 identical Pods.

```
  GNU nano 7.2                                                  deployment.yaml *
apiVersion: apps/v1
kind: Deployment
metadata:
  name: lab-deployment
  annotations:
    kubernetes.io/change-cause: "Initial Deployment 1.21"
    description: "This Lab is Great"
spec:
  replicas: 3
  selector:
    matchLabels:
      app: lab-app
  template:
    metadata:
      labels:
        app: lab-app
    spec:
      initContainers:
      - name: init-setup
        image: busybox:1.36
        command: ["sh", "-c", "echo \"Init Done\"; sleep 2"]
```

```
    containers:
    - name: nginx
      image: nginx:1.21
      ports:
      - containerPort: 80

      resources:
        requests:
          memory: "64Mi"
          cpu: "100m"
        limits:
          memory: "128Mi"
          cpu: "200m"

^G Help        ^O Write Out   ^W Where Is    ^K Cut
^X Exit        ^R Read File   ^\ Replace     ^U Pas
```

Apply:

Then I applied the new configuration by running the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment created
```

Rollout:

Then I ran the following command to confirm that the rollout was completed successfully.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

The output shows deployment "*lab-deployment*" successfully rolled out, which means the new replicas have been created and are in the Ready state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
```

Verifying describe:

Then I ran the following command and ran describe to get the original proof.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe deployment lab-deployment
```

The output shows that the Deployment is now maintaining 3 Pods and all Pods are successfully *Running* and *Available*.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe deployment lab-deployment
Name:                   lab-deployment
Namespace:              default
CreationTimestamp:      Tue, 17 Feb 2026 00:41:16 +0300
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
                        description: This Lab is Great
                        kubernetes.io/change-cause: Initial Deployment 1.21
Selector:               app=lab-app
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
```

5.5. Pod shell and logs access using deployment labels:

Pods created with deployments can usually change their names dynamically. So, the most practical way is to view logs and enter pod shells using the label selector instead of a specific Pod name. Here the Deployment label was *app=lab-app, so -l app=lab-app* was used to work on matching Pods.

View logs of any matching Pod by label:

Pod shell and logs access using deployment labels:

Pods created with deployments can usually change their names dynamically. So, the most practical way is to view logs and enter pod shells using the label selector instead of a specific Pod name. Here the Deployment label was *app=lab-app, so -l app=lab-app* was used to work on matching Pods.

View logs of any matching Pod by label:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs -l app=lab-app --tail=50
```

The output shows that the Pod template had an *initContainer (init-setup)* and a main container *(nginx*). *kubectl logs* shows the logs of the main container by default, which is normal behavior.

Then the log output shows *nginx/1.21.6* and start worker processes, which clearly proves that the *nginx* container has been started successfully and the application is running. This means that the Pods created by Deployment have worked properly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl logs -l app=lab-app --tail=50
Defaulted container "nginx" out of: nginx, init-setup (init)
/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/02/15 09:12:09 [notice] 1#1: using the "epoll" event method
2026/02/15 09:12:09 [notice] 1#1: nginx/1.21.6
2026/02/15 09:12:09 [notice] 1#1: built by gcc 10.2.1 20210110 (Debian 10.2.1-6)
2026/02/15 09:12:09 [notice] 1#1: OS: Linux 6.17.0-14-generic
2026/02/15 09:12:09 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1048576:1048576
2026/02/15 09:12:09 [notice] 1#1: start worker processes
2026/02/15 09:12:09 [notice] 1#1: start worker process 31
2026/02/15 09:12:09 [notice] 1#1: start worker process 32
2026/02/15 09:12:09 [notice] 1#1: start worker process 33
2026/02/15 09:12:09 [notice] 1#1: start worker process 34
```

Entering the shell by selecting a Pod using a label:

Using the following command, we collected the name of the first matching Pod using the label, without knowing the Pod name directly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
```

Then I gained shell access to that Pod using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it $POD -- sh
```

The output again shows Defaulted container "nginx"... because both *initContainer* and main container were there, and exec entered the nginx container by default. Then we got the shell prompt (#), which confirms that we entered the container environment.

When we tried to run the *kubectl* command inside the shell, we got *kubectl: not found*. This is not an error, but expected behavior. Because *kubectl* is not installed inside the nginx container image. To manage a Kubernetes cluster, *kubectl* always has to be run from the host machine (from where the cluster is being controlled).

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec -it $POD -- sh
Defaulted container "nginx" out of: nginx, init-setup (init)
# kubectl set image deployment/lab-deployment nginx=nginx:latest
sh: 1: kubectl: not found
# kubectl get rs -o wide
sh: 2: kubectl: not found
# kubectl describe rs -l app=lab-app | grep -E "Image|Replicas"
sh: 3: kubectl: not found
# exit
command terminated with exit code 1
```

5.6. Change config in Deployment YAML and monitor update:

In this step, I updated the application by changing the container image of the Deployment and observed the behavior of the Pods live during the rolling update. Earlier, the *nginx* image was *nginx:1.21*, now it has been changed to *nginx:latest*. When updating the image in the Deployment, Kubernetes does not delete the old Pods all at once; rather, it creates new Pods and gradually replaces the old Pods. This is the Rolling Update strategy.

First, I updated the image with the following command:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl set image deployment/lab-deployment nginx=nginx:latest
```

The output shows *deployment.apps/lab-deployment* image updated, which confirms that the Deployment spec has been updated.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl set image deployment/lab-deployment nginx=nginx:latest
deployment.apps/lab-deployment image updated
```

Then I used the following command to verify whether the rollout was successful.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

Here the deployment "*lab-deployment*" successfully rolled out shows that the Pod of the new version has been successfully created and is in the ready state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
```

Watching pods rolling update:

Checking Replica Sets:

The most important proof came from checking the Replica Set, in which I used the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get rs -o wide
```

The output shows two ReplicaSets:

• New *ReplicaSet (nginx:latest): DESIRED 1, READY 1*

• Old *ReplicaSet (nginx:1.21): DESIRED 0, READY 0*

This clearly shows that during the update a new *ReplicaSet* was created and the old *ReplicaSet* was scaled down. The deployment controller used the *RollingUpdate* strategy to deploy the new version and gradually remove the old version.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get rs -o wide
NAME                       DESIRED   CURRENT   READY   AGE   CONTAINERS   IMAGES         SELECTOR
lab-deployment-5d4846c4c   1         1         1       54s   nginx        nginx:latest   app=lab-app,pod-template-hash=5d4846c4c
lab-deployment-78ff9ddbd   0         0         0       35m   nginx        nginx:1.21     app=lab-app,pod-template-hash=78ff9ddbd
```

5.7. Rollback to previous application version:

Rollout history:

In this step, I have shown how to roll back to the previous stable version *(nginx:1.21*) using Deployment rollback if there is any problem after updating the nginx image to *nginx:latest.* The big advantage of Deployment is that it preserves the rollout revision, so you can quickly restore the previous version if needed.

First, I viewed the rollout history of the Deployment using the following command.

```
^Chasib@gitlab-24:~/Desktop/k8s-lab-diamondkubectl rollout history deployment/lab-deploymentnt
```

Here you can see *REVISION* and *CHANGE-CAUSE.* The output shows 1 Initial *Deployment 1.21*, which means the initial stable deployment is in the revision history (it is displayed in a report-friendly way because of the change-cause annotation).

```
^Chasib@gitlab-24:~/Desktop/k8s-lab-diamondkubectl rollout history deployment/lab-deploymentnt
deployment.apps/lab-deployment
REVISION  CHANGE-CAUSE
1         Initial Deployment 1.21
```

Rollback:

Then I rolled back to the previous revision using the following command to ensure that the rollout completed correctly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

The output shows deployment "*lab-deployment*" successfully rolled out, meaning the rollback process has completed successfully and the desired state is back to stable.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
```

Verifying which image running:

To verify which image is currently running, I described the *ReplicaSet* and checked the image using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe rs -l app=lab-app | grep -E "Image|Replicas"
```

The output shows *Replicas: 3 current / 3 desired* and *Image: nginx:1.21*, which means that nginx has reverted to *version 1.21* after the Deployment rollback and is running smoothly with the r*eplica count fixed* (3).

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe rs -l app=lab-app | grep -E "Image|Replicas"
Replicas:       3 current / 3 desired
    Image:      busybox:1.36
    Image:      nginx:1.21
```

The rollback was successful, so Deployment restored the previous stable *nginx version (nginx:1.21*), removed/scaled down the new version, and the application is running in a stable state again.

5.8. Setting CPU and Memory requests/limits + PoC (CPU limit vs Memory limit behavior):

Here *requests* = *tells* the scheduler how much resource the pod needs minimum (placement decision).

limits = *hard* cap; if CPU limit is reached, the container will be throttled, if memory limit is reached, it may *OOMKill*.

CPU PoC (throttling):

Pod name pick:

First, I selected a Pod using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
```

CPU spike generate:

Then I created an artificial CPU load inside the container using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec $POD -- sh -c "yes > /dev/null &"
```

The output contains *init-setup (init)*. It consumes CPU by running an infinite loop in the background.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl exec $POD -- sh -c "yes > /dev/null &"
Defaulted container "nginx" out of: nginx, init-setup (init)
```

Sleep:

I waited a while:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ sleep 10
```

Then I checked the metrics using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl top pod $POD
```

The output shows that the CPU usage is capped around the limit (*e.g. ~47m etc*). The important thing is that the container is not killed, but the CPU is capped. This is CPU throttling behavior. The application continues to run, but performance may degrade.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl top pod $POD
NAME                              CPU(cores)   MEMORY(bytes)
lab-deployment-5d4846c4c-46zwf    47m          10Mi
```

Ways to cleanup:

Deleting a pod will kill the background process.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete pod $POD
```

If you delete the Pod, the background yes process ends.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete pod $POD
pod "lab-deployment-5d4846c4c-46zwf" deleted from default namespace
```

Memory PoC (*OOMKilled*):

Creating exact *OOMKilled* proof with *mem-stress.yaml:*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano mem-stress.yaml
```

Here, *100m CPU* and *64Mi* memory are the minimum, and *200m CPU* and *128Mi* memory are the maximum limits.

```
  GNU nano 7.2                                                  mem-stress.yaml
apiVersion: v1
kind: Pod
metadata:
  name: mem-stress
spec:
  restartPolicy: Never
  containers:
  - name: oom
    image: alpine:3.19
    command: ["sh","-c"]
    args:
      - apk add --no-cache python3 >/dev/null;
        python3 -c 'a=[]; [a.append(bytearray(1024*1024)) for _ in range(150)]; import time; time.sleep(3600)'
    resources:
      limits:
        memory: "100Mi"
```

Apply:

To show what happens when the memory limit is exceeded, I used a separate mem-stress Pod, where memory usage is intentionally exceeded.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f mem-stress.yaml
pod/mem-stress created
```

Sleep:

I waited a while:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ sleep 10
```

Then I checked the Pod status using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod mem-stress -o wide
NAME         READY   STATUS             RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
mem-stress   0/1     ImagePullBackOff   0          31s   10.244.0.21   minikube   <none>           <none>
```

Here, I first saw *ImagePullBackOff* (image pull issue). Then we looked at the events section using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod mem-stress | tail -n 40
```

The output shows image pull error and container state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod mem-stress | tail -n 40
      --vm-hang
      1
    State:          Waiting
      Reason:       ErrImagePull
    Ready:          False
    Restart Count:  0
    Limits:
      memory:  100Mi
    Requests:
      memory:     100Mi
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-rq8gr (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
Volumes:
  kube-api-access-rq8gr:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
```

```
QoS Class:                   Burstable
Node-Selectors:              <none>
Tolerations:                 node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                             node.kubernetes.io/unreachable:NoExecute op=Exists for 300s
Events:
  Type     Reason     Age               From               Message
  ----     ------     ----              ----               -------
  Normal   Scheduled  42s               default-scheduler  Successfully assigned default/mem-stress to minikube
  Normal   Pulling    26s (x2 over 41s)  kubelet           spec.containers{stress}: Pulling image "progrium/stress"
  Warning  Failed     23s (x2 over 38s)  kubelet           spec.containers{stress}: Failed to pull image "progrium/stress": Error response from daemon: unsupporte
d manifest media type and no default available: application/vnd.docker.distribution.manifest.v1+prettyjws
  Warning  Failed     23s (x2 over 38s)  kubelet           spec.containers{stress}: Error: ErrImagePull
  Normal   BackOff    11s (x2 over 37s)  kubelet           spec.containers{stress}: Back-off pulling image "progrium/stress"
  Warning  Failed     11s (x2 over 37s)  kubelet           spec.containers{stress}: Error: ImagePullBackOff
```

Check reason:

Finally, I checked the container termination reason using the following command.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod mem-stress -o jsonpath='{.status.containerStatuses[0].state.terminated.reason}'
echo
OOMKilled
```

The output shows *OOMKilled*, meaning the runtime forcefully killed the container because it exceeded its memory limit.

Cleanup:

After the memory stress test was completed, I deleted the mem-stress Pod, so that the cluster would return to a clean state and there would be no unnecessary resource usage.

```
Swap:          3867        1986        1881
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete pod mem-stress
```

This output confirms that the mem-stress Pod was successfully deleted. Since this Pod was only used to demonstrate Memory limit behavior (*OOMKilled*), it was removed after the test was complete.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl delete pod mem-stress
pod "mem-stress" deleted from default namespace
```

5.9. Init Containers:

Adding an Init Container to Deployment YAML:

I have added an *init* container to my Deployment manifest, which executes a small setup step before nginx starts. Here I have used a simple command as a demonstration, which prints the message “Init Done” and gives a 2 second delay so that the *init* stage can be clearly observed.

```
    initContainers:
    - name: init-setup
      image: busybox:1.36
      command: ["sh", "-c", "echo \"Init Done\"; sleep 2"]
```

Pod Running State Verify:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -l app=lab-app
```

Here in the output below, 1/1 Running shows that the Pod is now fully ready. Since the *init* container executes first, getting to the Pod Running state means that the *init* stage has completed successfully and then the main nginx container has started.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -l app=lab-app
NAME                              READY   STATUS    RESTARTS   AGE
lab-deployment-57cbc9454d-tdftc   1/1     Running   0          23s
```

View the Init Containers Section from Pod Describe:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod lab-deployment-57cbc9454d-tdftc | grep -A5 "Init Containers"
```

Looking at the output, it can be seen that the Pod had an *init* container named *init-setup* attached and Kubernetes executed it.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod lab-deployment-57cbc9454d-tdftc | grep -A5 "Init Containers"
Init Containers:
  init-setup:
    Container ID:  docker://25dbe142eb486e8a9b50d4809442cf1f787ca2e5af93d6ca54d14d2490e965ea
    Image:         busybox:1.36
    Image ID:      docker-pullable://busybox@sha256:b9598f8c98e24d0ad42c1742c32516772c3aa2151011ebaf639089bd18c605b8
    Port:          <none>
```

5.10. Annotations in k8s:

Annotations are metadata key-values that store extra info on Kubernetes objects, but not for changing behavior like selector/label matching-mainly for documentation/automation/tools.

Use cases:

• Change history tracking (kubernetes.io/change-cause)

• CI/CD pipeline info (build ID, commit hash)

• ownership info (team name)

• Ingress controller configs (controller-specific annotations)

• external tools (Rancher/Helm) store extra metadata

In my deployment:

```
annotations:
  kubernetes.io/change-cause: "Initial Deployment 1.21"
  description: "This Lab is Great"
```

5.11. Report:

In this task I worked on Deployment update, rollback, resources limits, *initContainer* and annotations. First, I updated the application by changing the image of Deployment. Then I observed the rolling update process using the following commands.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
deployment "lab-deployment" successfully rolled out
```

During the update, Kubernetes created a new *ReplicaSet* and scaled down the old *ReplicaSet.* This is evidenced by the output below.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get rs -o wide
NAME                          DESIRED   CURRENT   READY   AGE   CONTAINERS   IMAGES         SELECTOR
lab-deployment-5d4846c4c      1         1         1       54s   nginx        nginx:latest   app=lab-app,pod-template-hash=5d4846c4c
lab-deployment-78ff9ddbd      0         0         0       35m   nginx        nginx:1.21     app=lab-app,pod-template-hash=78ff9ddbd
```

Then I rolled back to the previous version using *kubectl* rollout history and *kubectl* rollout undo and verified that the nginx image was back to the previous version by describing the *ReplicaSet.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe deployment lab-deployment
Name:                   lab-deployment
Namespace:              default
CreationTimestamp:      Tue, 17 Feb 2026 00:41:16 +0300
Labels:                 <none>
Annotations:            deployment.kubernetes.io/revision: 1
                        description: This Lab is Great
                        kubernetes.io/change-cause: Initial Deployment 1.21
Selector:               app=lab-app
Replicas:               3 desired | 3 updated | 3 total | 3 available | 0 unavailable
StrategyType:           RollingUpdate
MinReadySeconds:        0
RollingUpdateStrategy:  25% max unavailable, 25% max surge
```

In the next step, I set CPU/Memory requests and limits in the *Deployment YAML*. To test the CPU limit, I created a CPU stress inside the Pod and ran *kubectl* top pod to see if CPU usage was throttling.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl top pod $POD
NAME                                CPU(cores)   MEMORY(bytes)
lab-deployment-5d4846c4c-46zwf      47m          10Mi
```

That is, the container is not killed, but performance may be limited due to resource cap. Running a separate *mem-stress* pod to test memory limit, I saw that the container is *OOMKilled* when the memory limit is exceeded, which indicates hard kill behavior.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod mem-stress -o wide
NAME         READY   STATUS             RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
mem-stress   0/1     ImagePullBackOff   0          31s   10.244.0.21   minikube   <none>           <none>
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod mem-stress | tail -n 40
      --vm-hang
      1
    State:          Waiting
      Reason:       ErrImagePull
    Ready:          False
    Restart Count:  0
    Limits:
      memory:  100Mi
    Requests:
      memory:     100Mi
    Environment:  <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-rq8gr (ro)
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       False
  ContainersReady             False
  PodScheduled                True
Volumes:
  kube-api-access-rq8gr:
    Type:                    Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:  3607
    ConfigMapName:           kube-root-ca.crt
    Optional:                false
    DownwardAPI:             true
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pod mem-stress -o jsonpath='{.status.containerStatuses[0].state.terminated.reason}'
echo
OOMKilled
```

In addition, I added *initContainer* to do pre-work before the main container starts and showed proof with the *initContainer* section from the pod describe output.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod lab-deployment-57cbc9454d-tdftc | grep -A5 "Init Containers"
Init Containers:
  init-setup:
    Container ID:  docker://25dbe142eb486e8a9b50d4809442cf1f787ca2e5af93d6ca54d14d2490e965ea
    Image:         busybox:1.36
    Image ID:      docker-pullable://busybox@sha256:b9598f8c98e24d0ad42c1742c32516772c3aa2151011ebaf639089bd18c605b8
    Port:          <none>
```

Finally, we added deployment change history and description using annotations, which is important for documentation/traceability.

```
  annotations:
    kubernetes.io/change-cause: "Initial Deployment 1.21"
    description: "This Lab is Great"
```

Bonus - *ReplicaSets* involvement *PoC* (update process)

Verify old/new *ReplicaSet* by looking at *ReplicaSets* list:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get rs -o wide
```

Here are two *ReplicaSets* for the same Deployment:

- *lab-deployment-57cbc9454d: active/new ReplicaSet*, because *DESIRED/CURRENT/READY = 1* (meaning it is currently maintaining the Pod)
- *lab-deployment-5d6cdc75b8: old ReplicaSet*, because *DESIRED/CURRENT/READY = 0* (meaning it has been scaled down)

The *pod-template-hash* being different means that a new *ReplicaSet* has been generated due to a Pod template change.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get rs -o wide
NAME                          DESIRED   CURRENT   READY   AGE     CONTAINERS   IMAGES       SELECTOR
lab-deployment-57cbc9454d   1         1         1       11m     nginx        nginx:1.21   app=lab-app,pod-template-hash=57cbc9454d
lab-deployment-5d6cdc75b8   0         0         0       3h38m   nginx        nginx:1.21   app=lab-app,pod-template-hash=5d6cdc75b8
```

***ReplicaSet* describes Replicas and Image proofs:**

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe rs -l app=lab-app | grep -E "Replicas|Image"
```

This output shows that one *ReplicaSet* is now holding 1 desired/current (active). The other *ReplicaSet* is holding 0 desired/current (inactive/old). This is evidence of the update process - the deployment scaled down the old *RS* and kept the new *RS* running.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe rs -l app=lab-app | grep -E "Replicas|Image"
Replicas:       1 current / 1 desired
    Image:      busybox:1.36
    Image:      nginx:1.21
Replicas:       0 current / 0 desired
    Image:      busybox:1.36
    Image:      nginx:1.21
```

Proving revision increment from rollout history:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout history deployment/lab-deployment
```

The rollout history has changed from revision 1 to 2, indicating that the Deployment template has been updated and Kubernetes has tracked it as a new revision. Typically, revisions are incremented with each update, and each revision has a *ReplicaSet* associated with it, which allows for rollbacks.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout history deployment/lab-deployment
deployment.apps/lab-deployment
REVISION  CHANGE-CAUSE
1         Initial Deployment 1.21
2         Initial Deployment 1.21
```

Task 6 - k8s Secrets

Secret manifest is a quite similar to configMap . However, we use Secret to work with confidential application data. Kubernetes encode secrets in Base64 format.

1. Figure out the necessary Secret spec fields.

2. Create and apply a new Secret manifest. For example, it could be login and password to login to your app or something else.

3. With kubectl , get and describe your secret(s).

4. Decode your secret(s).

5. Update your Deployment to reference to your secret as environment variable.

6. Make sure that you are able to see your secret inside pod.

7. Question: Secrets objects are just a Base64 encoded data. We can't consider this as an encrypted data. What to do if you need to hide a sensitive data in real deployments? Propose your ideas.

8. Put the results into report.

Bonus: create a secret from json file. Hint: use ConfigMap logic for that. Repeat steps 3 - 6.

Solution:

6.1. Necessary Secret spec fields:

Secret manifest requires the following minimum:

• *apiVersion: v1*

• *kind: Secret*

• *metadata.name*

• *type (Opaque)*

• *data (Base64 encoded)* or *stringData* (if given plain text, *kubectl* will base64 it automatically)

6.2. Creating and applying a Secret manifest (username/password):

In this step I have created a Secret with a dummy username/password, so that the Secret creation and decoding process can be clearly shown.

Creating a Secret file:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano secret.yaml
```

The following config is placed in *secret.yaml.*

```
 GNU nano 7.2                                                  secret.yaml
apiVersion: v1
kind: Secret
metadata:
  name: app-secret
type: Opaque
stringData:
  username: admin
  password: passhasib123
```

Apply:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f secret.yaml
```

Output:

This output confirms that the secret named app-secret was successfully created in the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f secret.yaml
secret/app-secret created
```

6.3. Getting and describing secret(s):

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secrets
```

In the output below, *NAME* is the name of the Secret app-secret and *TYPE* is *Opaque* (generic secret type). *DATA* is 2 (username and password are 2 keys in total). Here the value of Secret is not shown, only the key count is shown.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secrets
NAME         TYPE     DATA   AGE
app-secret   Opaque   2      80s
```

Describing Secret:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe secret app-secret
```

The describe command does not show the actual value of the secret. It only shows the size (bytes) of each key. This is intentional security behavior so that sensitive data is not accidentally leaked to the terminal. Here password: 12 bytes and username: 5 bytes.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe secret app-secret
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  12 bytes
username:  5 bytes
```

6.4. Decode the secret(s):

To see the real value of the secret, you need to decode it with *Base64.*

Decode Username:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret -o jsonpath='{.data.username}' | base64 --decode
```

The decoded output yielded the username admin.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret -o jsonpath='{.data.username}' | base64 --decode
echo
admin
```

Decode password

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret -o jsonpath='{.data.password}' | base64 --decode
```

Here we see that the Secret data field was *Base64* encoded. After decoding, the original username and password were found.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret -o jsonpath='{.data.password}' | base64 --decode
echo
passhasib123
```

6.5. Using Secret as Environment Variable by Updating Deployment:

In this step, both the username and password key of the app-secret Secret are used as container environment variables of the Deployment. As a result, the secret value is available in the container runtime, but the raw value is not visible in the manifest or describe output**.**

Adding env to Deployment YAML:

```
        cpu: "200m"
      env:
        - name: APP_USERNAME
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: username
        - name: APP_PASSWORD
          valueFrom:
            secretKeyRef:
              name: app-secret
              key: password
```

env is used to define the environment variable of the container. Using *valueFrom.secretKeyRef*, we are asking to get the value from the Secret instead of giving it directly. *name: app-secret* is used to specify which Secret the value will come from and key: username/password is used to specify which key of the Secret will be used.

Apply + rollout check:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
```

In the output below, we can see that configured means the *Deployment* has been updated with the new configuration. Here, the *YAML* changes will create a new Pod.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment configured
```

Rollout Status Check:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

The output below shows that the rolling update has been completed and the old Pod has been terminated step by step. The new Pod has been started with the Secret-env configuration.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "lab-deployment" successfully rolled out
```

6.6. Verifying Secret inside Pod:

Pod name auto-select:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'env | grep -E "APP_USERNAME|APP_PASSWORD"'
```

In the above command *-l app=lab-app* is the label selector. The name of the first Pod is extracted by *jsonpath* and Pod name is stored in Variable *$POD*. The command can be run in the container shell by *kubectl exec*. The env shows all the environment variables and *grep* filters only *APP_USERNAME* and *APP_PASSWORD.*

APP_USERNAME and *APP_PASSWORD* are found in the output below. The output proves that Secret injection was successful.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'env | grep -E "APP_USERNAME|APP_PASSWORD"'
Defaulted container "nginx" out of: nginx, init-setup (init)
APP_USERNAME=admin
APP_PASSWORD=passhasib123
```

6.7. Securing Secrets in Production:

Kubernetes Secrets are only Base64 encoded by default, not encrypted.

(A) Encrypt secrets at *rest* (*etcd* encryption)

- Enabling *EncryptionConfiguration* on Kubernetes API server will store secrets in encrypted form in *etcd.*
- This is “cluster-side encryption”.

(B) External Secret Manager (recommended)

In production, secrets are typically stored in:

- HashiCorp Vault
- AWS Secrets Manager
- Azure Key Vault
- GCP Secret Manager

These are injected into Kubernetes using tools:

- External Secrets Operator (ESO)
- CSI Secrets Store Driver

(C) RBAC + least privilege

- Tightly control who can read the secret with RBAC
- Namespace isolation
- Separate service accounts

(D) Avoid plaintext secrets in GitOps

- Keeping secret yaml plaintext in the repo will leak
- alternative:
    - Sealed Secrets (Bitnami)
    - SOPS (Mozilla) + KMS/Vault

(E) Runtime security

- Node access hardening
- Audit logs
- Network Policies
- Image scanning + admission policies

6.8. Report:

In this task, I have successfully completed the entire process of creating, verifying, integrating and securing Kubernetes Secret. First, I created an *Opaque Secret* named *app-secret*, where the username and password keys are kept. Using the following two commands, I have confirmed that the *Secret* is created in the cluster and Kubernetes is only showing the data size (bytes) without showing the value directly, which is a security feature to prevent accidental exposure.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secrets
NAME         TYPE     DATA   AGE
app-secret   Opaque   2      80s
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe secret app-secret
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  12 bytes
username:  5 bytes
```

Then we verified the actual value of the Secret by *Base64* decoding, which proves that Kubernetes Secrets are only Base64 encoded by default, not encrypted.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret -o jsonpath='{.data.password}' | base64 --decode
echo
passhasib123
```

In the next step, I updated the Deployment manifest and injected the Secret as a container environment variable (*APP_USERNAME, APP_PASSWORD*). I used the following two to confirm that the rolling update was successful and the new Pods were created with the updated configuration.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment configured
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "lab-deployment" successfully rolled out
```

Then I selected a running Pod using the label selector and went inside the container with *kubectl exec* and checked the environment variable. There the Secret value was successfully found, which is runtime-level proof that Secret injection is working properly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'env | grep -E "APP_USERNAME|APP_PASSWORD"'
Defaulted container "nginx" out of: nginx, init-setup (init)
APP_USERNAME=admin
APP_PASSWORD=passhasib123
```

In the Bonus section, I created a Secret from a *JSON* file *(--from-file*), injected it into the Deployment, and verified it inside the Pod. The *JSON* content was verified by *Base64* decoding, which is a successful proof of using a file-based Secret.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'echo "$SECRET_JSON"'
Defaulted container "nginx" out of: nginx, init-setup (init)
{"apiKey":"SUF-980","user":"admin"}
```

Finally, we discussed Secret security, where Kubernetes Secrets are not encrypted by default, so encryption-at-rest, external secret manager (Vault/Cloud Secret Manager), RBAC least privilege, and GitOps-safe secret management should be used in production environments.

Bonus - Create secret from JSON file (*ConfigMap* logic):

Creating *JSON* file:

Here a JSON file is created and it is used as secret data.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ cat > secret.json <<'EOF'
{"apiKey":"SUF-980","user":"admin"}
EOF
```

Creating secret from file:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl create secret generic app-secret-json --from-file=secret.json
```

Opaque secret is created with *generic. --from-file* stores the entire file base64 encoded in secret and the key is the file name (*secret.json*). The output shows evidence of *json* file creation.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl create secret generic app-secret-json --from-file=secret.json
error: failed to create secret secrets "app-secret-json" already exists
```

Get and describe:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret-json
```

The Secret has been created in the Output. *DATA = 1* indicates that there is a key and *Type = Opaque.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret-json
NAME              TYPE     DATA   AGE
app-secret-json   Opaque   1      38h
```

Describe:

The output shows that the password in the *Data* section was 12 bytes and the username was 5 bytes.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe secret app-secret
Name:         app-secret
Namespace:    default
Labels:       <none>
Annotations:  <none>

Type:  Opaque

Data
====
password:  12 bytes
username:  5 bytes
```

Decode JSON:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret-json -o jsonpath='{.data.secret\.json}' | base64 --decode; echo
```

The output shows that the Base64 data inside the Secret has been decoded and the original JSON has been found. This proves that the Secret was Base64 encoded.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get secret app-secret-json -o jsonpath='{.data.secret\.json}' | base64 --decode; echo
{"apiKey":"SUF-980","user":"admin"}
```

Deployment Apply (*JSON* Secret inject):

Added to Deployment env:

```
          - name: APP_PASSWORD
            valueFrom:
              secretKeyRef:
                name: app-secret
                key: password

          - name: SECRET_JSON
            valueFrom:
              secretKeyRef:
                name: app-secret-json
                key: secret.json


^G Help         ^O Write Out    ^W Where Is     ^K Cu
^X Exit         ^R Read File    ^\ Replace      ^U Pa
```

Apply:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
```

The output shows that the Deployment has been updated and the *JSON* secret has been injected into the new Pod.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment configured
```

Rollout verify:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

The output shows that the rolling update was successful and the new Pod is ready with the *JSON* secret.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "lab-deployment" successfully rolled out
```

Verify inside pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'echo "$SECRET_JSON"'
```

The output shows that the Secret value was found inside the Container and the File-based Secret injection was successful.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'echo "$SECRET_JSON"'
Defaulted container "nginx" out of: nginx, init-setup (init)
{"apiKey":"SUF-980","user":"admin"}
```

Task 7 - k8s ConfigMap

ConfigMap is Kubernetes manifest to store application configuration setting in two ways: key-value pairs as environment variables and text (like JSON) data as dedicated file into container filesystem. We usually use ConfigMap to store non sensitive data as plain text.

1. Figure out the necessary ConfigMap spec fields.

2. Modify your Deployment manifest to set up some app configuration via environment variables.

3. Create a new ConfigMap manifest. In data spec, put some app configuration as key-value pair (it could be the same as in previous exercise). In the Deployment.Pod spec add the connection to key-value pair from ConfigMap yaml file.

4. Create a new file like config.json file and put some json data into.

5. Create a new one configMap manifest. Connect configMap yaml file with config.json file to read the data from it.

6. Update your Deployment to add Volumes and VolumeMounts .

7. With kubectl , check the ConfigMap details. Make sure that you see the data as plain text.

8. Check the filesystem inside app container to show the loaded file data on the specified path.

9. Put the results into report.

Solution:

7.1. Necessary spec fields of *ConfigMap*:

The minimum fields required to create a ConfigMap are:

• *apiVersion: v1* (Kubernetes API version)

• *kind: ConfigMap* (object type)

• *metadata.name* (*ConfigMap* name)

• *data:* (configuration data in key-value format)

ConfigMap is mainly used to store non-sensitive configuration. Unlike Secret, it is not sensitive data, it is stored in plain text.

7.2-3. Creating *ConfigMap* (key-value) and using it in Deployment:

Creating *ConfigMap YAML*:

Before creating any object in Kubernetes, a *YAML* manifest needs to be created.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano configmap-env.yaml
```

Here it means:

• *name: app-config-env* will create a *ConfigMap* with this name

• *data:* will be used to store the key-value configuration

• *APP_MODE* and *APP_REGION* are used as container environment variables later

```
 GNU nano 7.2                                                  configmap-env.yaml *
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-env
data:
  APP_MODE: "production"
  APP_REGION: "ru"
```

Apply *ConfigMap*:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f configmap-env.yaml
```

The following command is used to create or update objects in Kubernetes cluster as per YAML manifest. Here we have created *ConfigMap* as per *configmap-env.yaml* file.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f configmap-env.yaml
configmap/app-config-env configured
```

Adding env to Deployment (Deployment YAML update):

Here we have added env: section inside the container spec of Deployment so that the key-value pair of *ConfigMap* is injected as container environment variable.

```
            - name: APP_MODE
              valueFrom:
                configMapKeyRef:
                  name: app-config-env
                  key: APP_MODE
            - name: APP_REGION
              valueFrom:
                configMapKeyRef:
                  name: app-config-env
                  key: APP_REGION

^G Help        ^O Write Out    ^W Where Is     ^K Cu
^X Exit        ^R Read File    ^\ Replace      ^U Pa
```

Here, the use of *configMapKeyRef* fetches the value from *configMap* and uses the name: *app-config-env* to access the *ConfigMap*. The use of key: fetches the key from the *ConfigMap* and in the container runtime, these will be env variables named *APP_MODE* and *APP_REGION.*

Deployment update apply:

This command is used to apply the new configuration after updating the deployment manifest. It triggers a rolling update.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
```

As you can see below, the Deployment has been successfully updated. New Pods will be created with the updated configuration.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment configured
```

Rollout status check:

This command is used to verify whether the rolling update was completed successfully.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

The output shows that Kubernetes is gradually terminating old Pods and creating new Pods. As a result, the rollout is complete when all new Pods are ready. This proves that the new Pods have been created with the configuration added to the *ConfigMap.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "lab-deployment" successfully rolled out
```

Verify environment variables inside Pod:

This command dynamically fetches the name of a Pod based on the label (app=lab-app). It does not require hardcoding. *kubectl exec* is used to run the command inside the Pod and env shows all the environment variables. grep is used to filter specific variables.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'env | grep -E "APP_MODE|APP_REGION"'
```

Here we see that the key-value pair injected from *ConfigMap* is successfully available as an environment variable in the container runtime.

This means that the *ConfigMap* was in the cluster, the Deployment was updated, the Pod was recreated, and the configuration reached the Container environment.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'env | grep -E "APP_MODE|APP_REGION"'
Defaulted container "nginx" out of: nginx, init-setup (init)
APP_MODE=production
APP_REGION=ru
```

7.4. Create a *config.json* file to store JSON data:

Create *config.json:*

In this step, I created a separate configuration file called *config.json,* which stores some of the application's settings in JSON format. In reality, many apps read settings from JSON/YAML config files, so this file is created before mounting the *ConfigMap* as a “file”.

For this, I used *cat > config.json <<'EOF' ... EOF*-this is a heredoc method, where multiple lines can be written to the file at once and the formatting is correct. This reduces manual typing errors.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ cat > config.json <<'EOF'
{
  "appName": "lab-nginx",
  "environment": "production",
  "featureFlags": {
    "newUI": true,
    "beta": false
  }
}
EOF
```

Verify:

To verify that the file was indeed created correctly, we ran *cat config.json.* The cat command displays the contents of the file in the terminal. This serves as proof that the *JSON* content is as expected.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ cat config.json
{
  "appName": "lab-nginx",
  "environment": "production",
  "featureFlags": {
    "newUI": true,
    "beta": false
  }
}
```

7.5. Creating and applying *ConfigMap* manifest with *config.json* data:

Now the goal was to put the configuration like *config.json* in Kubernetes *ConfigMap*, so that the configuration can be managed centrally without rebuilding the container image. Here I created a *ConfigMap* named app-config-file, and put the entire JSON content as *config.json* key inside the *data: section.*

The reason for creating *configmap-file.yaml* is that keeping *ConfigMap* in manifest form makes version control easier, and the same config can be applied *repeatedly.ConfigMap Manifest: configmap-file.yaml*

```
 GNU nano 7.2                                    configmap-file.yaml *
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config-file
data:
  config.json: |
    {
      "appName": "lab-nginx",
      "environment": "production",
      "featureFlags": {
        "newUI": true,
        "beta": false
      }
    }
```

Apply:

Then, *kubectl apply -f* is run to create/update this YAML cluster. The apply command uses a declarative approach to match the Kubernetes state even if the same YAML is run repeatedly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f configmap-file.yaml
```

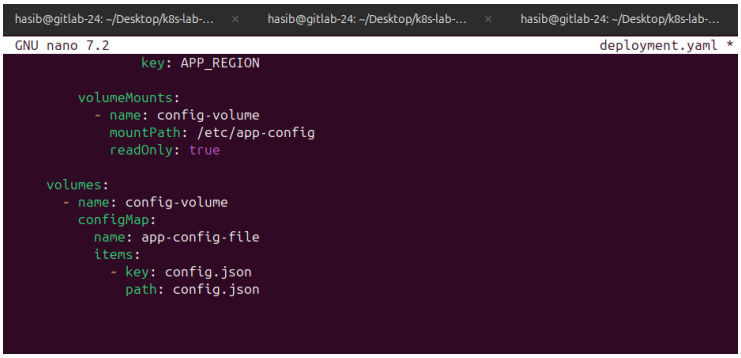
This output means that the app-config-file *ConfigMap* has been successfully applied to the cluster. Configured means that it has been updated if it exists before, otherwise it has been created.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f configmap-file.yaml
configmap/app-config-file configured
```

7.6. Adding Volumes and *VolumeMounts* by updating the Deployment (*ConfigMap* file mount:

***volumeMounts* inside the container:**

This tells the container where to look for the config file. I gave the *mountPath to /etc/app-config*, and set *readOnly* to true to prevent accidental writes. This tells Kubernetes that the volume named config-volume will actually come from the *app-config-file ConfigMap*, and only the *config.json* key will be mounted as a file.



Apply:

Then the following command was run to apply the Deployment update, because the manifest had changed. After applying, Kubernetes started a rolling update, where it removed the old Pods and created Pods with the new config.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f deployment.yaml
deployment.apps/lab-deployment configured
```

Rollout:

Then I used the following command to check if the rollout was completed properly. Because *ConfigMap* mount changes are fully reflected only when a new Pod is created.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
```

The output shows that the Deployment has been updated, new Pods have been created, and the application is in a stable state with the *ConfigMap* volume mounted.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "lab-deployment" successfully rolled out
```

7.7. Check *ConfigMap* details with *kubectl*:

In this step I have verified that *ConfigMap* data is visible in plain text form in the cluster (not hidden like Secret). For this I first looked at all *ConfigMap* lists to make sure that the objects are present.

List *ConfigMaps:*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get configmaps
NAME               DATA   AGE
app-config-env     2      37h
app-config-file    1      31h
kube-root-ca.crt   1      2d1h
```

Describe env *ConfigMap:*

There are both *app-config-env (ENV* keys) and app-config-file (file content). In the DATA column, you can see 2 keys in env config, and 1 key in file config (*config.json*).

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe configmap app-config-env
```

*AP*P_MODE and *APP_REGION* appear as plain text in the output, which is proof that *ConfigMap* is non-sensitive plaintext storage.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe configmap app-config-env
Name:         app-config-env
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_MODE:
----
production

APP_REGION:
----
ru


BinaryData
====

Events:  <none>
```

Describe file *ConfigMap*:

Then I used *kubectl* describe because the describe command displays the data part of the *ConfigMap* in a readable format.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe configmap app-config-file
```

Here the output shows the entire *config.json* content in plain text format.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe configmap app-config-file
Name:         app-config-file
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
config.json:
----
{
  "appName": "lab-nginx",
  "environment": "production",
  "featureFlags": {
    "newUI": true,
    "beta": false
  }
}


BinaryData
====

Events:  <none>
```

YAML view:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get configmap app-config-file -o yaml
```

The output contains the full JSON text with *config.json:* | inside the data: section, confirming that it was stored in plaintext in the *ConfigMap* cluster. The keys are visible in *app-config-env.*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get configmap app-config-file -o yaml
apiVersion: v1
data:
  config.json: |
    {
      "appName": "lab-nginx",
      "environment": "production",
      "featureFlags": {
        "newUI": true,
        "beta": false
      }
    }
kind: ConfigMap
metadata:
  annotations:
    kubectl.kubernetes.io/last-applied-configuration: |
      {"apiVersion":"v1","data":{"config.json":"{\n  \"appName\": \"lab-nginx\",\n  \"environment\": \"production\",\n  \"featureFlags\": {\n    \"newUI\": true,\n
    \"beta\": false\n  }\n}\n"},"kind":"ConfigMap","metadata":{"annotations":{},"name":"app-config-file","namespace":"default"}}
  creationTimestamp: "2026-02-15T19:32:04Z"
  name: app-config-file
  namespace: default
  resourceVersion: "126136"
  uid: b9df30d2-fdf8-4524-95fe-de4f6883d62b
```

7.8. View mounted files in container filesystem:

I used label to collect the name of any running Pod of Deployment. Because Deployment can have multiple replicas, I used label selector (-l app=lab-app) instead of hardcoded pod name.

I used *kubectl get pods -l app=lab-app* to find all matching Pods created by Deployment. Using label selector makes it easy to find the correct Pods, no need to use hardcoded names. I used *-o*

jsonpath=... to find only the name of the first Pod. This provides specific fields without showing the entire table, which is convenient for automation. I *used kubectl exec -it "$POD"* to run commands inside the container of the selected Pod. Through this, the container environment can be directly tested. I used *ls -l /etc/app-config* to verify whether the file exists in the mount path. This shows whether the *ConfigMap* has been correctly added as a volume.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'ls -l /etc/app-config && echo "----" && cat /etc/app-config/config.json'
```

config.json -> ..data/config.json This output shows that *config.json* is actually a symbolic link *(symlink).* When Kubernetes mounts a *ConfigMap* volume, it uses an internal directory called *..data/* instead of a direct file. This ensures that Kubernetes can perform atomic updates. That is, when a *ConfigMa*p is updated, Kubernetes creates a new data directory and points the *symlink* to the new directory. This allows the updated configuration to be available without restarting the container.

The output of *cat /etc/app-config/config.json* shows the *JSON* content in plain text. This proves that the *ConfigMap* was successfully mounted inside the Pod, the file system path is working properly, and the application will be able to read configuration data from this path.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'ls -l /etc/app-config && echo "----" && cat /etc/app-config/config.json'
Defaulted container "nginx" out of: nginx, init-setup (init)
total 0
lrwxrwxrwx 1 root root 18 Feb 17 03:21 config.json -> ..data/config.json
----
{
  "appName": "lab-nginx",
  "environment": "production",
  "featureFlags": {
    "newUI": true,
    "beta": false
  }
}
```

7.9. Report:

In this Task, the application configuration is set in two ways using Kubernetes *ConfigMap* – once as an environment variable and once as a file mounted on the container filesystem.

In the first step, a *ConfigMap* named app-config-env is created, where *APP_MODE* and *APP_REGION* are stored as key-value pairs. Then, the deployment manifest is added with an env section and *configMapKeyRef* is used, so that these values are injected as environment variables when the container is started. Going inside the container and running the env command, we see *APP_MODE=production* and *APP_REGION=ru*. This proves that the environment variable injection from the ConfigMap is working successfully.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ POD=$(kubectl get pods -l app=lab-app -o jsonpath='{.items[0].metadata.name}')
kubectl exec -it "$POD" -- sh -c 'env | grep -E "APP_MODE|APP_REGION"'
Defaulted container "nginx" out of: nginx, init-setup (init)
APP_MODE=production
APP_REGION=ru
```

In the second step, a *config.json* file is created and the *JSON* configuration is stored there. Then a *ConfigMap* named app-config-file is created and stored in that *JSON* content data section.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl rollout status deployment/lab-deployment
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 2 out of 3 new replicas have been updated...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
Waiting for deployment "lab-deployment" rollout to finish: 1 old replicas are pending termination...
deployment "lab-deployment" successfully rolled out
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get configmaps
NAME               DATA   AGE
app-config-env     2      37h
app-config-file    1      31h
kube-root-ca.crt   1      2d1h
```

Deployment has been updated to add volumes and *volumeMounts,* so that the *ConfigMap* is mounted to the */etc/app-config* path. After the rollout is complete, checking the container filesystem shows that the */etc/app-config/config.json* file is present and the *JSON* content inside it is readable in plain text. This proves that the *ConfigMap* file-mount was successful.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe configmap app-config-env
Name:         app-config-env
Namespace:    default
Labels:       <none>
Annotations:  <none>

Data
====
APP_MODE:
----
production

APP_REGION:
----
ru


BinaryData
====

Events:  <none>
```

An important point is that the data in the *ConfigMap* is stored in plain text in the Kubernetes cluster. Running the above and below commands will show the entire data in readable form.



This is different from Secret, because Secret data is *Base64* encoded (although it is not true encryption), but *ConfigMap* stores the configuration in full plaintext.

Task 8 – k8s Namespace

Namespace Kubernetes Manifest is designed for different projects and deployment environments isolation. With Namespaces we can separate App 1 deployment from App 2 deployment, manage (and isolate) cluster resources for them, define users list to have access either to App 1 or to App 2 deployment. Using Namespaces , we also can define a different environments like DEV, TEST, STAGE. In that way, Namespaces is a required feature for a real Kubernetes production clusters.

1. Figure out the necessary Namespace spec fields.

2. Create a two different Namespaces in your k8s cluster.

3. Using kubectl , get and describe your Namespaces.

4. Deploy two different applications in two different Namespaces with kubectl. By the way, it's acceptable even just to deploy the same objects (same app you used before) in the different Namespaces but with unique resources names.

5. With kubectl , get and describe pods from different Namespaces witn -n flag.

6. Does Namespaces isolate objects from each other?

7. How does accessing Service objects located on different Namespaces change your work with this k8s object kind?

8. Put the results into report.

Bonus: install and show a practical example of https://github.com/ahmetb/kubectx kubens usage.

Solution:

8.1. Necessary Namespace spec fields

Minimum required fields of namespace manifest:

- *apiVersion: v1*
- *kind: Namespace*
- *metadata.name*

Example:

```
  GNU nano 7.2                                                  namespaces.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev1
---
apiVersion: v1
kind: Namespace
metadata:
  name: dev2
```

8.2. Create two different Namespaces:

In this step, I created two separate namespaces in the *cluster: dev1* and *dev2*. I used the *YAML* manifest (declarative) method because it provides version control and maintains the desired state even if the same file is applied repeatedly.

Creating the manifest:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano namespaces.yaml
```

The namespace spec has been input in the *namespaces.yaml* file.

```
  GNU nano 7.2                                                  namespaces.yaml
apiVersion: v1
kind: Namespace
metadata:
  name: dev1
---
apiVersion: v1
kind: Namespace
metadata:
  name: dev2
```

Apply:

The following command creates/updates objects in Kubernetes according to *YAML*. The output shows *namespace/dev1* created and *namespace/dev2* created, meaning that two namespaces have been successfully created in the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get ns dev1 dev2
```

Once a namespace is created, resources deployed within that namespace remain separate from other namespaces.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f namespaces.yaml
namespace/dev1 created
namespace/dev2 created
```

8.3. Using *kubectl,* get and describe Namespaces:

View specific namespaces:

The namespace list is displayed with *kubectl get ns* . Here *STATUS = Active* means the namespace is ready and usable. AGE shows how long ago the namespace was created, this works well as proof in reports.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get ns dev1 dev2
```

The output shows that *STATUS = Active* means the namespace is ready and usable. *AGE* shows how long ago the namespace was created.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get ns dev1 dev2
NAME   STATUS   AGE
dev1   Active   12m
dev2   Active   2m7s
```

namespaces list:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get namespaces
```

Here we see not only the *dev1* and *dev2* we created, but also the Kubernetes system namespaces (*kube-system, kube-public, kube-node-lease, kubernetes-dashboard*). This proves that the cluster is running with standard components and the namespaces are registered correctly.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get namespaces
NAME                   STATUS   AGE
default                Active   2d2h
dev1                   Active   14m
dev2                   Active   4m35s
kube-node-lease        Active   2d2h
kube-public            Active   2d2h
kube-system            Active   2d2h
kubernetes-dashboard   Active   2d2h
```

namespace details describe:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe namespace dev1
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe namespace dev1
Name:         dev1
Labels:       kubernetes.io/metadata.name=dev1
Annotations:  <none>
Status:       Active

No resource quota.

No LimitRange resource.
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe namespace dev2
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe namespace dev2
Name:         dev2
Labels:       kubernetes.io/metadata.name=dev2
Annotations:  <none>
Status:       Active

No resource quota.

No LimitRange resource.
```

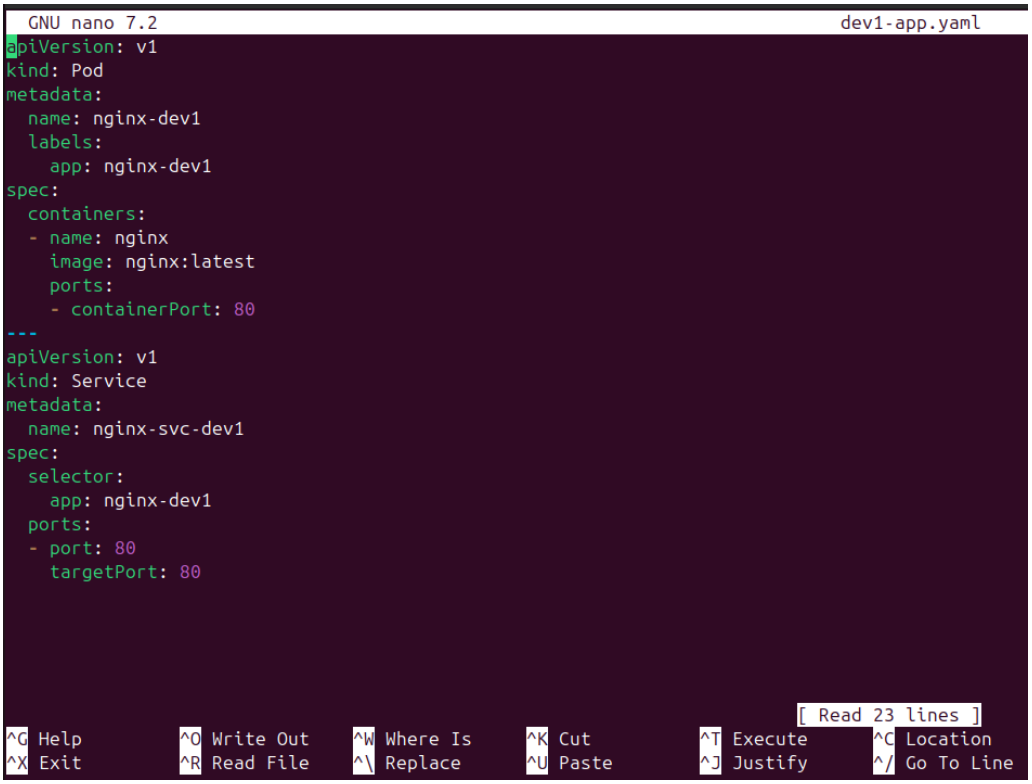
Using the *kubectl* describe namespace command displays detailed information about a specific namespace. *Namespaces dev1* and *dev2* were in Active status, which means the namespace was created correctly and is currently available for use.

The output also shows that no *ResourceQuota* is set, meaning that no quota has been set to limit the number of CPU, memory, or pods in this namespace yet. Similarly, LimitRa*nge* is not set, meaning that no per-pod default CPU or memory limit has been set.

8.4. Deploy two different applications in two different Namespaces:

In this step, I have deployed the same *nginx* app in dev1 and dev2 namespaces with different resource names. Because in Kubernetes, resources with the same name conflict within the same

namespace, but the same name is theoretically possible in different namespaces. Still, as per the assignment requirement, I have given a unique name so that clear separation can be understood.



Deploy the app in the dev1 namespace:

Using *-n dev1* here means that all Kubernetes objects (such as Pods, Services, etc.) in the manifest will be created or updated in the specific dev1 namespace. If no namespace is specified, the object is created in the default namespace, so *-n dev1* is used to keep the specific environment separate.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -n dev1 -f dev1-app.yaml
```

The output shows *pod/nginx-dev1* created that a new nginx Pod has been successfully created inside the dev1 namespace. And *service/nginx-svc-dev1* unchanged means that the Service object already existed in the dev1 namespace, so its current configuration remains unchanged without being created anew.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -n dev1 -f dev1-app.yaml
pod/nginx-dev1 created
service/nginx-svc-dev1 unchanged
```

Deploy the app in the dev2 namespace:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano dev2-app.yaml
```

The namespace spec for dev2 is inserted below.

```
  GNU nano 7.2                                                dev2-app.yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-dev2
  labels:
    app: nginx-dev2
spec:
  containers:
  - name: nginx
    image: nginx:latest
    ports:
    - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc-dev2
spec:
  selector:
    app: nginx-dev2
  ports:
  - port: 80
    targetPort: 80
```

Here, the dev2 namespace is specifically targeted using *-n dev2*. This ensures that all objects in the manifest are created in the dev2 environment, not in the default namespace.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -n dev2 -f dev2-app.yaml
```

The output *pod/nginx-dev2* created shows that a new *nginx Pod* has been successfully created inside the dev2 namespace. Also, *service/nginx-svc-dev2* created indicates that a new Service has also been created in this namespace, which is associated with the Pod.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -n dev2 -f dev2-app.yaml
pod/nginx-dev2 created
service/nginx-svc-dev2 created
```

8.5. Using *kubectl*, get and describe Pods from different Namespaces (*-n flag*):

If the namespace is different, you must use the *-n* flag to see the same type of resource (such as Pod) because Kubernetes searches in the default namespace by default.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev1
```

This command shows the Pod list of dev1 namespace. Here *nginx-dev1 Pod* is in Running state, which means the application is running successfully in dev1 namespace.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev1
NAME         READY   STATUS    RESTARTS   AGE
nginx-dev1   1/1     Running   0          26m
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev2
```

Here we see the Pods in the dev2 namespace. *nginx-dev2* is in the Running state, which proves that dev1 and dev2 are running in separate environments.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev2
NAME         READY   STATUS    RESTARTS   AGE
nginx-dev2   1/1     Running   0          94s
```

Wide View:

I also verified the Pod IP and Node information using the following commands.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev1 -o wide
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev1 -o wide
NAME         READY   STATUS    RESTARTS   AGE   IP            NODE       NOMINATED NODE   READINESS GATES
nginx-dev1   1/1     Running   0          26m   10.244.0.81   minikube   <none>           <none>
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev2 -o wide
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev2 -o wide
NAME         READY   STATUS    RESTARTS   AGE    IP            NODE       NOMINATED NODE   READINESS GATES
nginx-dev2   1/1     Running   0          2m1s   10.244.0.82   minikube   <none>           <none>
```

Here, the Pod IPs are different:

dev1 = 10.244.0.81

dev2 = 10.244.0.82

This shows that the Pods in the two namespaces are logically isolated but are using the same cluster network.

Describe Pod:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod nginx-dev1 -n dev1
Name:             nginx-dev1
Namespace:        dev1
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Tue, 17 Feb 2026 07:01:05 +0300
Labels:           app=nginx-dev1
Annotations:      <none>
Status:           Running
IP:               10.244.0.81
IPs:
  IP:  10.244.0.81
Containers:
  nginx:
    Container ID:   docker://ea0109db1b98aacffcf7a0954d381704d27d34077158f28d2b389309acd26674
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Tue, 17 Feb 2026 07:01:14 +0300
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-j5rfw (ro)
```

```
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-j5rfw:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  27m   default-scheduler  Successfully assigned dev1/nginx-dev1 to minikube
  Normal  Pulling    27m   kubelet            spec.containers{nginx}: Pulling image "nginx:latest"
  Normal  Pulled     27m   kubelet            spec.containers{nginx}: Successfully pulled image "nginx:latest" in 2.626s (2.626s including waiting). Image size: 16
0850673 bytes.
  Normal  Created    27m   kubelet            spec.containers{nginx}: Container created
  Normal  Started    27m   kubelet            spec.containers{nginx}: Container started
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod nginx-dev2 -n dev2
Name:             nginx-dev2
Namespace:        dev2
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Tue, 17 Feb 2026 07:26:04 +0300
Labels:           app=nginx-dev2
Annotations:      <none>
Status:           Running
IP:               10.244.0.82
IPs:
  IP:  10.244.0.82
Containers:
  nginx:
    Container ID:   docker://60fb1b760fea7b400579f4d1a492bf774c3e75f0581b2d1a63e9845eb4548474
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Tue, 17 Feb 2026 07:26:11 +0300
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-6scpv (ro)
```

```
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-6scpv:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason     Age    From               Message
  ----    ------     ----   ----               -------
  Normal  Scheduled  2m37s  default-scheduler  Successfully assigned dev2/nginx-dev2 to minikube
  Normal  Pulling    2m33s  kubelet            spec.containers{nginx}: Pulling image "nginx:latest"
  Normal  Pulled     2m30s  kubelet            spec.containers{nginx}: Successfully pulled image "nginx:latest" in 2.734s (2.734s including waiting). Image size: 1
60850673 bytes.
  Normal  Created    2m30s  kubelet            spec.containers{nginx}: Container created
  Normal  Started    2m30s  kubelet            spec.containers{nginx}: Container started
```

The output below shows that the two Pods are in different namespaces (dev1 and dev2), ensuring logical isolation. Each Pod has a different IP, meaning they are running with a separate network

identity. The Events section shows that the Pod was successfully scheduled, image pulled, and container started.

8.6. Does Namespace isolate objects from each other?

Yes, Namespace logically isolates most Kubernetes objects.

Isolated objects:

- Pods
- Services
- Deployments
- *ConfigMaps*
- Secrets

This means:

- Pods created on dev1 cannot be seen from dev2 with kubectl get pods.
- Resources with the same name can exist in two namespaces (no name collisions).

However, some objects are cluster-wide:

- Nodes
- *PersistentVolume*
- Namespace
- *ClusterRole / ClusterRoleBinding*
- *CRD*

Namespace isolation works at the logical level, not the physical cluster level.

8.7. Accessing Services across namespaces (DNS changes):

The rules for using the Service DNS name change because the namespace is different.

From Different namespace (Fully Qualified Domain Name):

Format:

<service>.<namespace>.svc.cluster.local

My PoC (dev2 pod to dev1 service access):

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run test-dns -n dev2 --image=curlimages/curl --restart=Never --command -- \
  sh -c "curl -s nginx-svc-dev1.dev1.svc.cluster.local | head"
```

pod/test-dns created and Logs check found nginx welcome page. Here, Service of dev1 namespace is accessed from Pod of dev2 namespace. It would not work if FQDN was not used.

This proves that cross-namespace communication is possible even with Namespace isolation. However, DNS has to be fully qualified.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl run test-dns -n dev2 --image=curlimages/curl --restart=Never --command -- \
  sh -c "curl -s nginx-svc-dev1.dev1.svc.cluster.local | head"
pod/test-dns created
```

8.8. Report:

In this task, separate environments or projects (dev1 and dev2) within the same cluster are logically isolated using Kubernetes *Namespac*e. The main purpose of this task was to demonstrate that it is possible to control resource separation, naming isolation and cross-namespace communication using Namespace.

First, two *Namespaces* named dev1 and dev2 are created. The *Namespace* manifest uses *apiVersion: v1*, *kind: Namespace* and *metadata.name*, which are the minimum fields required to create a Namespace. The following command shows *Status: Active*, which confirms that the namespaces have been created correctly in both clusters and are ready to use.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get ns dev1 dev2
NAME   STATUS   AGE
dev1   Active   12m
dev2   Active   2m7s
```

Next, the following two commands were used to verify that dev1 and dev2 were registered as separate namespaces.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get namespaces
NAME                   STATUS   AGE
default                Active   2d2h
dev1                   Active   14m
dev2                   Active   4m35s
kube-node-lease        Active   2d2h
kube-public            Active   2d2h
kube-system            Active   2d2h
kubernetes-dashboard   Active   2d2h
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe namespace dev1
Name:         dev1
Labels:       kubernetes.io/metadata.name=dev1
Annotations:  <none>
Status:       Active

No resource quota.

No LimitRange resource.
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe namespace dev2
Name:         dev2
Labels:       kubernetes.io/metadata.name=dev2
Annotations:  <none>
Status:       Active

No resource quota.

No LimitRange resource.
```

There is no *ResourceQuota* or *LimitRange* set in the output, meaning that the namespaces are currently in an unrestricted state. This shows that the namespace has been created but no resource policy has been applied yet.

The same nginx application is then deployed to two different namespaces. Pods named *nginx-dev1* are created in the dev1 namespace and *nginx-dev2* in the dev2 namespace. The *-n* flag is used during deployment so that the resource is created in a specific namespace. The following commands ensure that the corresponding Pod in each namespace is in the Running state. This clearly shows that the resource is being managed separately according to the Namespace.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev1
NAME         READY   STATUS    RESTARTS   AGE
nginx-dev1   1/1     Running   0          26m
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -n dev2
NAME         READY   STATUS    RESTARTS   AGE
nginx-dev2   1/1     Running   0          94s
```

Detailed analysis of the Pods was done using *kubectl describe pod -n <namespace>* . The output shows that the Namespace field shows dev1 and dev2 respectively, each Pod has a different IP, and the Events section shows that the Pods were successfully scheduled and started. This proves that the Pods in the two namespaces are logically isolated from each other.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod nginx-dev1 -n dev1
Name:             nginx-dev1
Namespace:        dev1
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Tue, 17 Feb 2026 07:01:05 +0300
Labels:           app=nginx-dev1
Annotations:      <none>
Status:           Running
IP:               10.244.0.81
IPs:
  IP:  10.244.0.81
Containers:
  nginx:
    Container ID:   docker://ea0109db1b98aacffcf7a0954d381704d27d34077158f28d2b389309acd26674
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Tue, 17 Feb 2026 07:01:14 +0300
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-j5rfw (ro)
```

```
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-j5rfw:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason     Age   From               Message
  ----    ------     ----  ----               -------
  Normal  Scheduled  27m   default-scheduler  Successfully assigned dev1/nginx-dev1 to minikube
  Normal  Pulling    27m   kubelet            spec.containers{nginx}: Pulling image "nginx:latest"
  Normal  Pulled     27m   kubelet            spec.containers{nginx}: Successfully pulled image "nginx:latest" in 2.626s (2.626s including waiting). Image size: 16
0850673 bytes.
  Normal  Created    27m   kubelet            spec.containers{nginx}: Container created
  Normal  Started    27m   kubelet            spec.containers{nginx}: Container started
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe pod nginx-dev2 -n dev2
Name:             nginx-dev2
Namespace:        dev2
Priority:         0
Service Account:  default
Node:             minikube/192.168.49.2
Start Time:       Tue, 17 Feb 2026 07:26:04 +0300
Labels:           app=nginx-dev2
Annotations:      <none>
Status:           Running
IP:               10.244.0.82
IPs:
  IP:  10.244.0.82
Containers:
  nginx:
    Container ID:   docker://60fb1b760fea7b400579f4d1a492bf774c3e75f0581b2d1a63e9845eb4548474
    Image:          nginx:latest
    Image ID:       docker-pullable://nginx@sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
    Port:           80/TCP
    Host Port:      0/TCP
    State:          Running
      Started:      Tue, 17 Feb 2026 07:26:11 +0300
    Ready:          True
    Restart Count:  0
    Environment:    <none>
    Mounts:
      /var/run/secrets/kubernetes.io/serviceaccount from kube-api-access-6scpv (ro)
```

```
Conditions:
  Type                        Status
  PodReadyToStartContainers   True
  Initialized                 True
  Ready                       True
  ContainersReady             True
  PodScheduled                True
Volumes:
  kube-api-access-6scpv:
    Type:                     Projected (a volume that contains injected data from multiple sources)
    TokenExpirationSeconds:   3607
    ConfigMapName:            kube-root-ca.crt
    Optional:                 false
    DownwardAPI:              true
QoS Class:                    BestEffort
Node-Selectors:               <none>
Tolerations:                  node.kubernetes.io/not-ready:NoExecute op=Exists for 300s
                              node.kubernetes.io/unreachable:NoExecute op=Exists for 300s

Events:
  Type    Reason     Age    From               Message
  ----    ------     ----   ----               -------
  Normal  Scheduled  2m37s  default-scheduler  Successfully assigned dev2/nginx-dev2 to minikube
  Normal  Pulling    2m33s  kubelet            spec.containers{nginx}: Pulling image "nginx:latest"
  Normal  Pulled     2m30s  kubelet            spec.containers{nginx}: Successfully pulled image "nginx:latest" in 2.734s (2.734s including waiting). Image size: 1
60850673 bytes.
  Normal  Created    2m30s  kubelet            spec.containers{nginx}: Container created
  Normal  Started    2m30s  kubelet            spec.containers{nginx}: Container started
```

Analyzing namespace isolation, we see that Pods, Services, Deployments, *ConfigMaps,* and Secrets act as *namespaced* resources. That is, resources in dev1 are not directly visible to dev2, and resources with the same name can be in different namespaces. However, some resources such

as Node, *PersistentVolume, ClusterRole*, etc. are cluster-wide, which are not limited by namespaces. Therefore, Namespace provides logical isolation, not physical cluster isolation. Namespace also plays an important role in service access. To access a service within the same namespace, you only need to use the service name. But to access from a different namespace, you need to use the Fully Qualified Domain Name (FQDN), such as *<service-name>.<namespace>.svc.cluster.local*. In practical tests, dev1 service was accessed from dev2 namespace and the nginx welcome page was successfully obtained. This proves that cross-namespace communication is possible, but DNS needs to be fully qualified.

Bonus - *kubectx/kubens* practical usage (PoC):

This section shows a practical demonstration of how to simplify namespace switching by installing the *kubectx* and *kubens* tools. This tool is very useful for quickly switching namespaces in large Kubernetes clusters without having to repeatedly use the *-n flag*.

Install Process:

First, the GitHub repository is cloned:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ sudo git clone https://github.com/ahmetb/kubectx /opt/kubectx
```

The output shows that this package already exists.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ sudo git clone https://github.com/ahmetb/kubectx /opt/kubectx
[sudo] password for hasib:
fatal: destination path '/opt/kubectx' already exists and is not an empty directory.
```

Then a symbolic link is created so that it can be used system-wide:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ sudo ln -sf /opt/kubectx/kubens /usr/local/bin/kubens
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ sudo ln -sf /opt/kubectx/kubectx /usr/local/bin/kubectx
```

Now *kubens* and *kubectx* will work as global commands.

List All Namespaces Using kubens:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubens
default
dev1
dev2
kube-node-lease
kube-public
kube-system
kubernetes-dashboard
```

This shows all the namespaces in the cluster in list form. This is a simple and fast alternative to *kubectl get ns.*

Switch namespace quickly:

Switch to dev1:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubens dev1
```

Active namespace is "dev1" found in the output. Notice that only pods in the dev1 namespace are shown, even without using any *-n dev1 flag*.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubens dev1
Context "minikube" modified.
Active namespace is "dev1".
```

Get pods:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods
```

Only resources from the dev2 namespace are visible here.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods
NAME         READY   STATUS    RESTARTS   AGE
nginx-dev1   1/1     Running   0          58m
```

Switch to dev2:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubens dev2
```

It can be observed here that only pods in the dev1 namespace are shown even without using any *-n dev2 flag*.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubens dev2
Context "minikube" modified.
Active namespace is "dev2".
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods
NAME         READY   STATUS      RESTARTS   AGE
nginx-dev2   1/1     Running     0          34m
test-dns     0/1     Completed   0          18m
```

As soon as I switch the namespace, *the* output shows the resources of that namespace.

Previously, the "*resources not found*" problem occurred because the namespace switch did not revert to the default namespace.

To check the current namespace:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl config view --minify -o jsonpath='{..namespace}'; echo
dev2
```

Currently the dev2 namespace is active.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl config set-context --current --namespace=default
Context "minikube" modified.
```

Context "minikube" modified is found in the output.

Task 9 - k8s RBAC

RBAC (Role-based access control) is a system for allocating access rights to various objects in a Kubernetes cluster.

1. Figure out the necessary ServiceAccount , ClusterRole and ClusterRoleBinding specs fields.

2. Create a new ServiceAccount dedicated to your app deployment.

3. Create ClusterRole and ClusterRoleBinding manifests, connect them to the custom ServiceAccount then.

4. With kubectl , get and describe your Service Accounts, Roles and Bindings.

5. Provide a Poc that your custom ServiceAccount has a different permissions to cluster resources rather than default one.

6. Questions: why to create a custom ServiceAccounts is a good practice? Why it's useful? What is the difference between Roles and ClusterRoles and their Bindings ?

7. Put the results into report.

Solution:

9.1. Necessary spec fields (ServiceAccount / ClusterRole / ClusterRoleBinding):

ServiceAccount

- *apiVersion: v1*
- *kind: ServiceAccount*
- *metadata.name*
- *metadata.namespace*

ClusterRole

- *apiVersion: rbac.authorization.k8s.io/v1*
- *kind: ClusterRole*
- *metadata.name*
- *rules[]:*
    - *apiGroups*
    - *resources*
    - *verbs*

ClusterRoleBinding

- *apiVersion: rbac.authorization.k8s.io/v1*
- *kind: ClusterRoleBinding*
- *metadata.name*
- *subjects[]:*
    - *kind: ServiceAccount*
    - *name*
    - *namespace*
- *roleRef:*
    - *kind: ClusterRole*
    - *name*
    - *apiGroup: rbac.authorization.k8s.io*

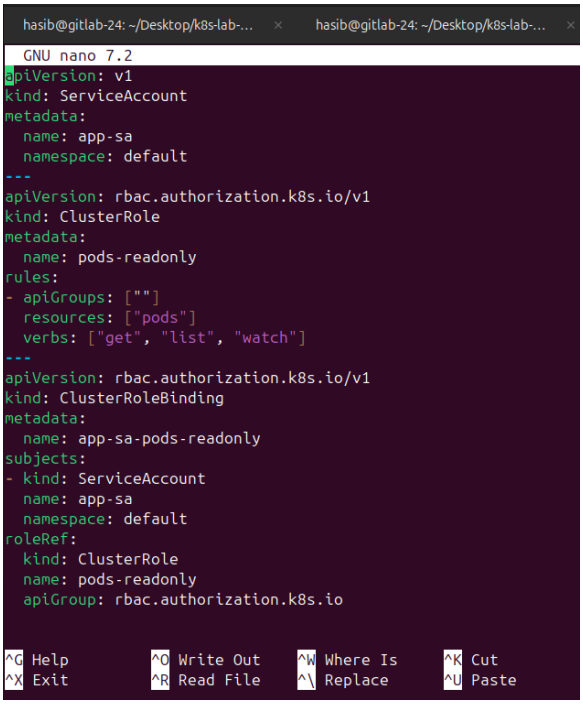
9.2. Create a new ServiceAccount dedicated to the app deployment

In this step, I created a new *ServiceAccount* called *app-sa*, which can be attached to the Pods of the app/deployment in the future. This means that when the Pod accesses the Kubernetes API, it will use the *app-sa* identity instead of the default *ServiceAccount* and its permissions will be controlled by RBAC.

9.3. Create *ClusterRole* + *ClusterRoleBinding*, connect to the *ServiceAccount*:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ nano rbac.yaml
```

In this step, I created a *ClusterRole* called *pods-readonly,* which only has permission to read pods (get, list, watch). Then I created a *ClusterRoleBinding* and bound it to the *app-sa ServiceAccount*. As a result, *app-sa* will now be able to read pods in the cluster, but will not have privileges like write/delete. This is the principle of least privilege.



Apply:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f rbac.yaml
```

The output shows that *serviceaccount/app-sa* created means that a new *ServiceAccount* has been successfully created, which is used as the app identity. *clusterrole...pods-readonly* created means that a read-only permission set has been created for pods. *clusterrolebinding...* created This permission set (*pods-readonly*) is now associated with the *app-sa ServiceAccount*. As a result, the RBAC setup is complete and *app-sa* now gets access according to the specified rules.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f rbac.yaml
serviceaccount/app-sa created
clusterrole.rbac.authorization.k8s.io/pods-readonly created
clusterrolebinding.rbac.authorization.k8s.io/app-sa-pods-readonly created
```

9.4. With *kubectl*, get and describe your *ServiceAccounts*, Roles and Bindings:

To list *ServiceAccounts*:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get sa -n default
```

This command lists all the *ServiceAccounts* in the default namespace. Here we see that there are two *ServiceAccounts*-default (created automatically by Kubernetes) and *app-sa* (created by us custom). This means that our custom identity has been created successfully.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get sa -n default
NAME      AGE
app-sa    30h
default   2d3h
```

***ClusterRole* View:**

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get clusterrole pods-readonly
```

This command checks if the specified ClusterRole (pods-readonly) exists. Here we see that the role was created successfully and the creation time is displayed. This means that the permission set has been registered in the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get sa -n default
NAME      AGE
app-sa    30h
default   2d3h
```

ClusterRoleBinding View:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get clusterrole pods-readonly
```

This command shows that the *app-sa-pods-readonly* binding has been created and is associated with the *pods-readonly ClusterRole*. This means that the binding between the *ServiceAccount* and the Role is effective.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get clusterrole pods-readonly
NAME            CREATED AT
pods-readonly   2026-02-15T23:06:08Z
```

Describe resources:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe sa app-sa -n default
```

Here is the detailed metadata for the *ServiceAccount.* It is in the default namespace and has the image pull secret attached.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get clusterrolebinding app-sa-pods-readonly
NAME                ROLE                        AGE
app-sa-pods-readonly   ClusterRole/pods-readonly   30h
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe clusterrole pods-readonly
```

This output is very important. Here we see:

- Resource = pods
- Allowed verbs = get, list, watch

This role can only read pods, it cannot delete or create them. This is read-only access.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe clusterrole pods-readonly
Name:         pods-readonly
Labels:       <none>
Annotations:  <none>
PolicyRule:
  Resources  Non-Resource URLs  Resource Names  Verbs
  ---------  -----------------  --------------  -----
  pods       []                 []              [get list watch]
```

Describe *ClusterRoleBinding:*

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe clusterrolebinding app-sa-pods-readonly
```

Here we can clearly see that *pods-readonly ClusterRole* and bound to *app-sa ServiceAccount*. *app-sa* now has *pods-readonly* permission.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe clusterrolebinding app-sa-pods-readonly
Name:         app-sa-pods-readonly
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  pods-readonly
Subjects:
  Kind            Name    Namespace
  ----            ----    ---------
  ServiceAccount  app-sa  default
```

9.5. PoC - Custom *ServiceAccount* vs Default *ServiceAccount*:

Check permission for custom SA:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i list pods --as=system:serviceaccount:default:app-sa
```

This is checking if *app-sa* can list pods? Answer: yes. This means the permission is working.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i list pods --as=system:serviceaccount:default:app-sa
yes
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i delete pods --as=system:serviceaccount:default:app-sa
```

Here it is seen that there is no delete permission. That is, the role is limited (least privilege).

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i delete pods --as=system:serviceaccount:default:app-sa
no
```

Compare with default SA:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i list pods --as=system:serviceaccount:default:default
no
```

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i delete pods --as=system:serviceaccount:default:default
no
```

Default ServiceAccount has no pods permissions. app-sa has extra read permissions, default does not.

9.6.

Why custom ServiceAccounts is good practice / useful?

• **Least privilege (Principle of Least Privilege):** The app only gets the permissions it needs.

• *Reduces blast radius:* If a pod is compromised, the attacker won't have access to everything.

• **Auditing + accountability***:* It's clear which workload is running with which permissions.

• **Separation of concerns***:* Having different SAs for different apps / environments makes management easier.

• **Avoid using default SA:** Default SA is usually used indeterminately/riskily; it's best practice to avoid it in production.

Difference between *Roles* vs *ClusterRoles*, and *Bindings:*

• **Role:** namespace-scoped (applies only to resources in a specific namespace)

• **ClusterRole:** cluster-scoped (can apply to the entire cluster; can grant cluster-wide grants even to namespace resources)

Bindings:

• **RoleBinding**: binds a *Role/ClusterRole* within a namespace (namespace-limited scope)

• **ClusterRoleBinding:** binds cluster-wide (can grant permissions to all namespaces / cluster scopes)

Easy memory:

• *Role/RoleBinding* - namespace level

• *ClusterRole/ClusterRoleBinding* - cluster level

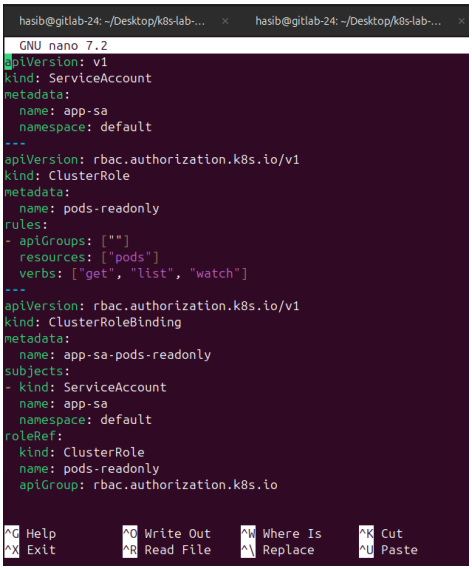
9.7. Report:

The purpose of this task was to create a custom *ServiceAccount* using Kubernetes RBAC, assign it a specific *ClusterRole*, and assign permissions via *ClusterRoleBinding*. Later, *kubectl* auth can-*i* was used to verify that the custom *ServiceAccount* was getting different permissions than the default *ServiceAccount*.

Creating *ServiceAccount*, *ClusterRole*, and *ClusterRoleBinding*:

First, the following resources were defined by creating the rbac.yaml file:

- *ServiceAccount: app-sa*
- *ClusterRole: pods-readonly*
- *ClusterRoleBinding: app-sa-pods-readonly*



The command was used to apply and this output proves that three RBAC objects were successfully created in the cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl apply -f rbac.yaml
serviceaccount/app-sa created
clusterrole.rbac.authorization.k8s.io/pods-readonly created
clusterrolebinding.rbac.authorization.k8s.io/app-sa-pods-readonly created
```

***ServiceAccount* Verification:**

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get sa -n default
NAME      AGE
app-sa    30h
default   2d3h
```

Here we see that the *app-sa* custom ServiceAccount has been successfully created and exists in the default namespace.

Verify ClusterRole:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get clusterrole pods-readonly
NAME            CREATED AT
pods-readonly   2026-02-15T23:06:08Z
```

This *ClusterRole* is only granting get, list, watch permission for the pods resource.

That is, it was *read-only* access.

ClusterRoleBinding verification:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl describe clusterrolebinding app-sa-pods-readonly
Name:         app-sa-pods-readonly
Labels:       <none>
Annotations:  <none>
Role:
  Kind:  ClusterRole
  Name:  pods-readonly
Subjects:
  Kind            Name    Namespace
  ----            ----    ---------
  ServiceAccount  app-sa  default
```

It is clearly seen here that the *pods-readonly ClusterRole* is bound to the app-sa *ServiceAccount.*

PoC - Custom *ServiceAccount* vs Default *ServiceAccount*:

Custom *ServiceAccount* permission check:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i list pods --as=system:serviceaccount:default:app-sa
yes
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i delete pods --as=system:serviceaccount:default:app-sa
no
```

app-sa can list pods but cannot delete them. It has limited *read-only* permission.

Default *ServiceAccount* permission check:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i list pods --as=system:serviceaccount:default:default
no
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl auth can-i delete pods --as=system:serviceaccount:default:default
no
```

The Default *ServiceAccount* is not getting any pods permissions, but the *app-sa* is getting specific *read-only* permissions. This proves that the RBAC configuration is working successfully.

Bonus - Rancher On real complicated production Kubernetes clusters, it's very important to have a centralized GUI based system to easily access, control and monitor k8s deployments. Install any k8s container management platform and show several actions related to cluster control in practice.

Rancher is one of the most popular solution.

Solution:

Why Rancher is needed:

It is difficult to manage everything in a production-grade Kubernetes cluster with just *kubectl.* So, using a centralized GUI-based platform (like Rancher) makes it easier to access cluster, monitor, manage workloads, logs, scaling, namespace/RBAC control from one place.

My environment evidence (as per my system):

My environment already had *minikube* and GitLab containers running. GitLab was using *port 80*, so I mapped Rancher to *8081/8443* instead of the *default 80/443*.

Rancher installation:

Remove old container:

The following command removes the Rancher container if it is present and ensures a clean state.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker rm -f rancher 2>/dev/null || true
rancher
```

Rancher container run:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker run -d \
  --name rancher \
  --restart=unless-stopped \
  -p 8081:80 \
  -p 8443:443 \
  --privileged \
  rancher/rancher:latest
```

A container ID was returned, which proves that Rancher was successfully started.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker run -d \
  --name rancher \
  --restart=unless-stopped \
  -p 8081:80 \
  -p 8443:443 \
  --privileged \
  rancher/rancher:latest
5c68bcb2f8ec8fcbb923b577855efa88d41edcfdb8d8e63025d5d900841c2858
```

Evidence:

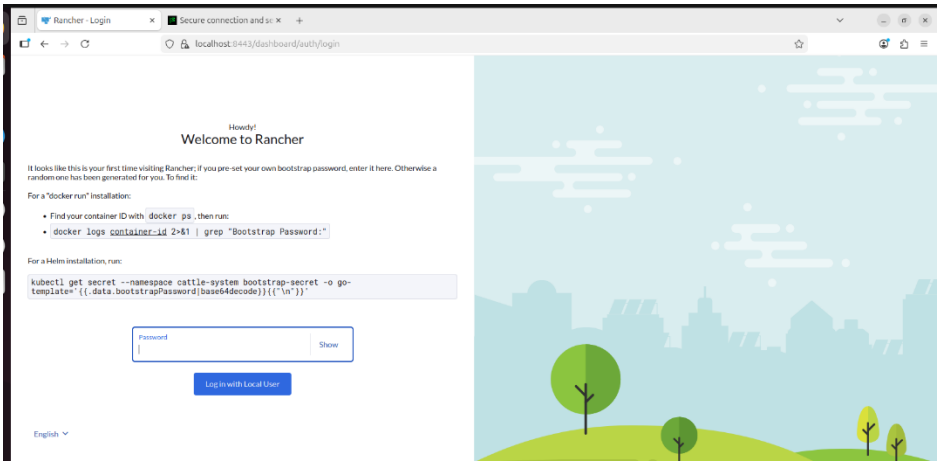
```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker ps
```

The output shows:

- 0.0.0.0:8081->80/tcp
- 0.0.0.0:8443->443/tcp

Rancher exposed HTTP and HTTPS ports.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS                    PORTS                                                
```



Admin Password Reset:

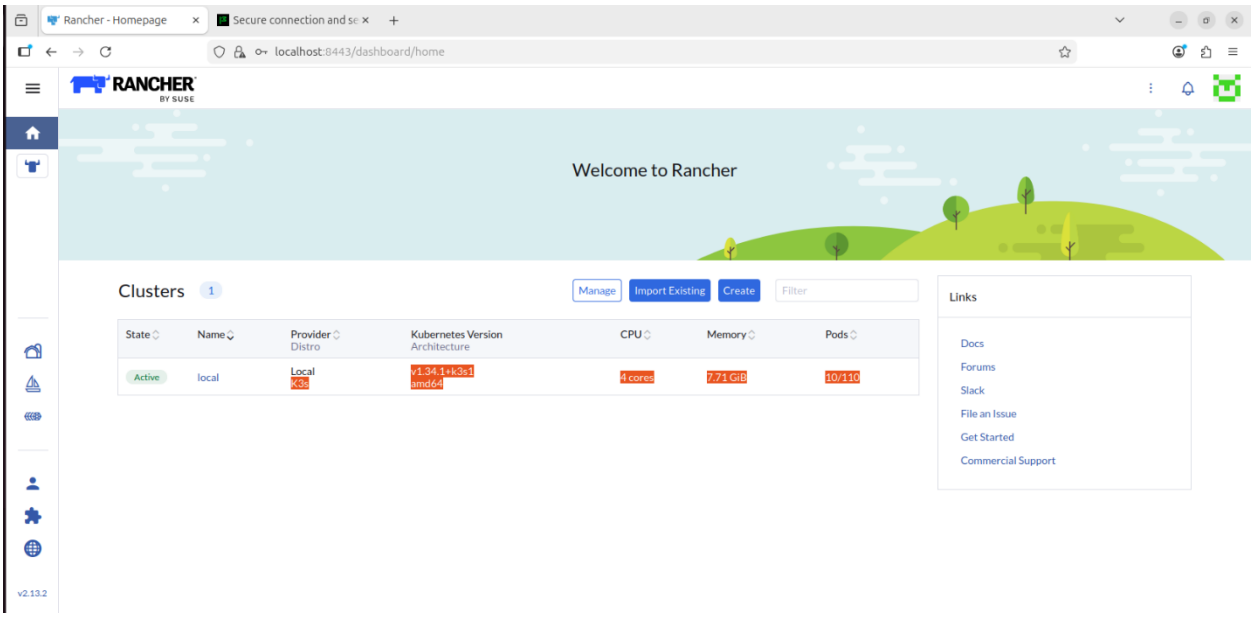
The password has been reset from the container for the first login:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker exec -it rancher reset-password
```

GUI login was successfully completed using the password obtained from the output.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ docker exec -it rancher reset-password
INFO[0000] Running in single server mode, will not peer connections
New password for default admin user (user-lzdzs):
ToM3mJUk1lBb14dtULsq
```

Successfully login:

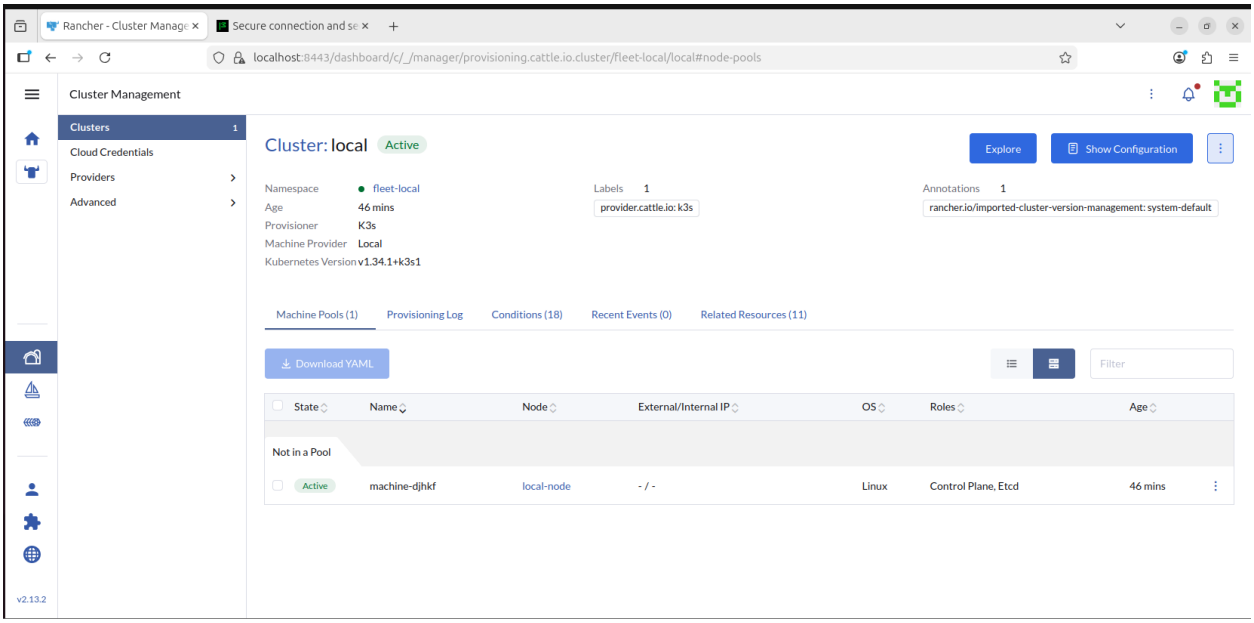


Practical GUI actions - Cluster control in Rancher:

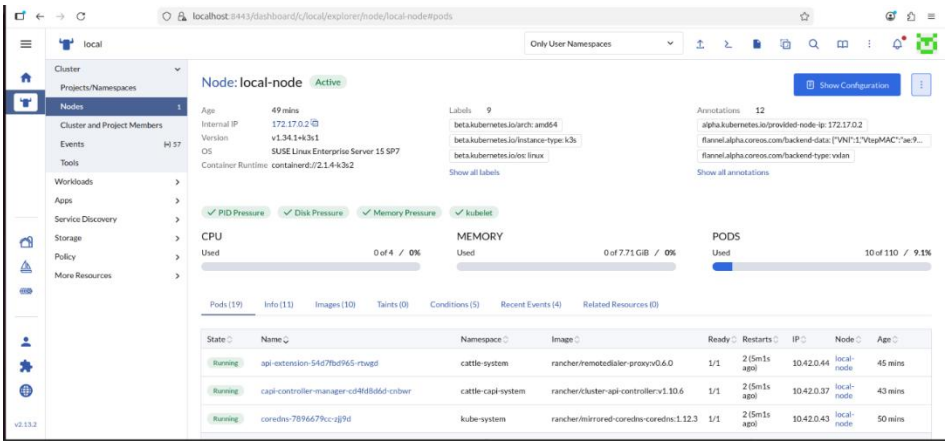
Action A: Cluster view and Nodes monitoring

Switched from *Cluster Explorer* to *local*.

Found *Cluster-local* status (*Active*) in *Nodes* tab.



Kubernetes version, resource usage (CPU/RAM):



CLI verification:

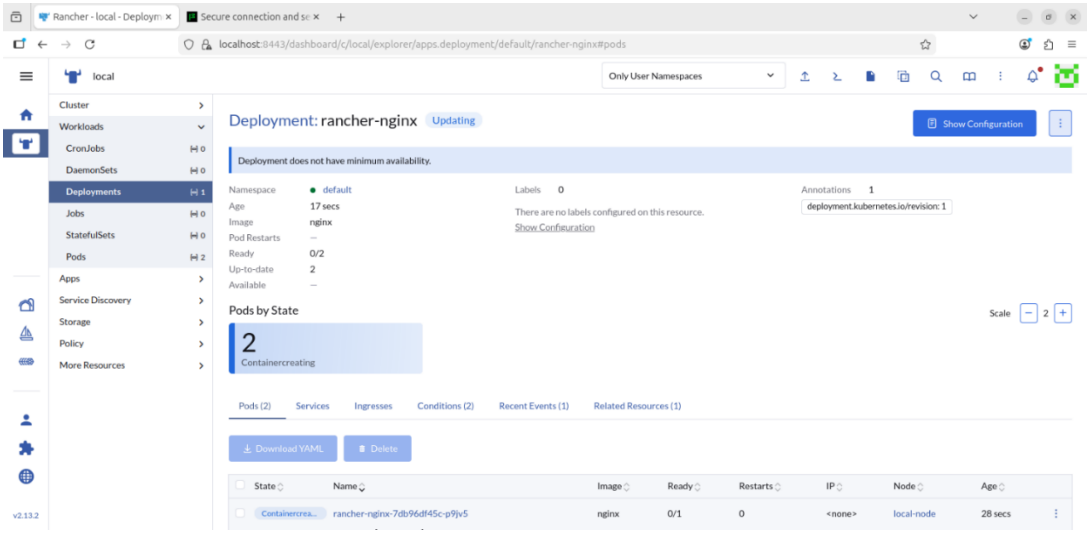
```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes
```

The same cluster status was seen in both the GUI and CLI.

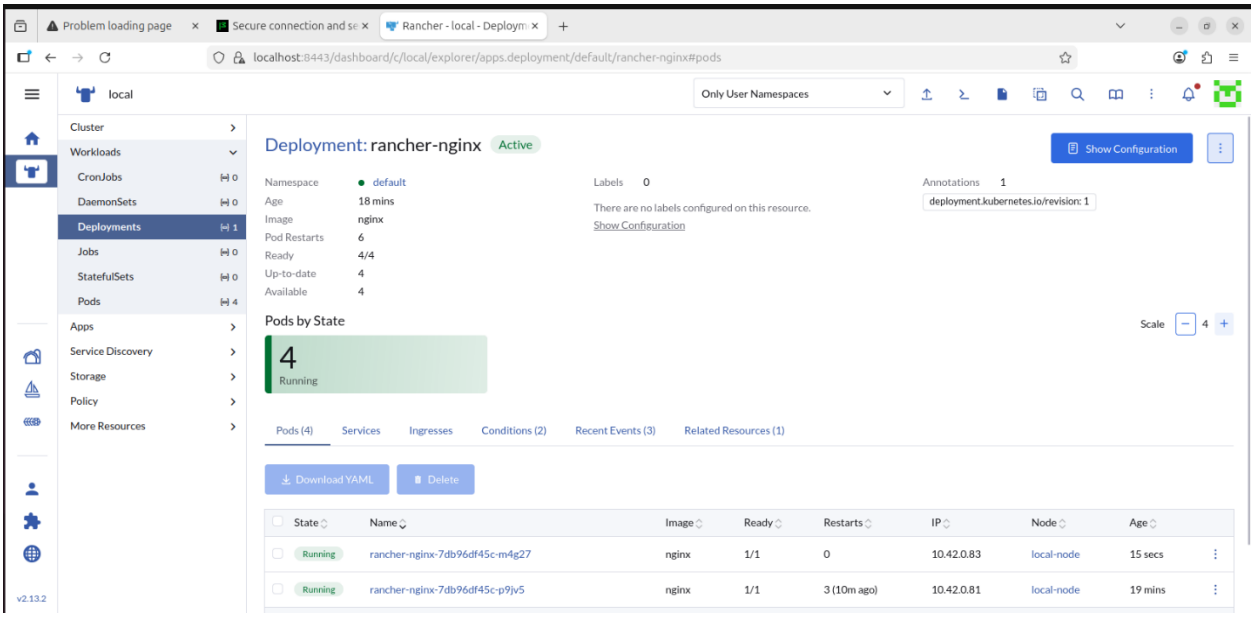
```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get nodes
NAME       STATUS   ROLES           AGE   VERSION
minikube   Ready    control-plane   36h   v1.35.0
```

Action B: Scale deployment (via GUI)

Go from *Workloads* to *Deployment*. Then go to *Scale* and take replicas 2 to 4.



Pods by State: 4



Then pods increase is visible in the UI.

CLI evidence add with kubectl get pods:

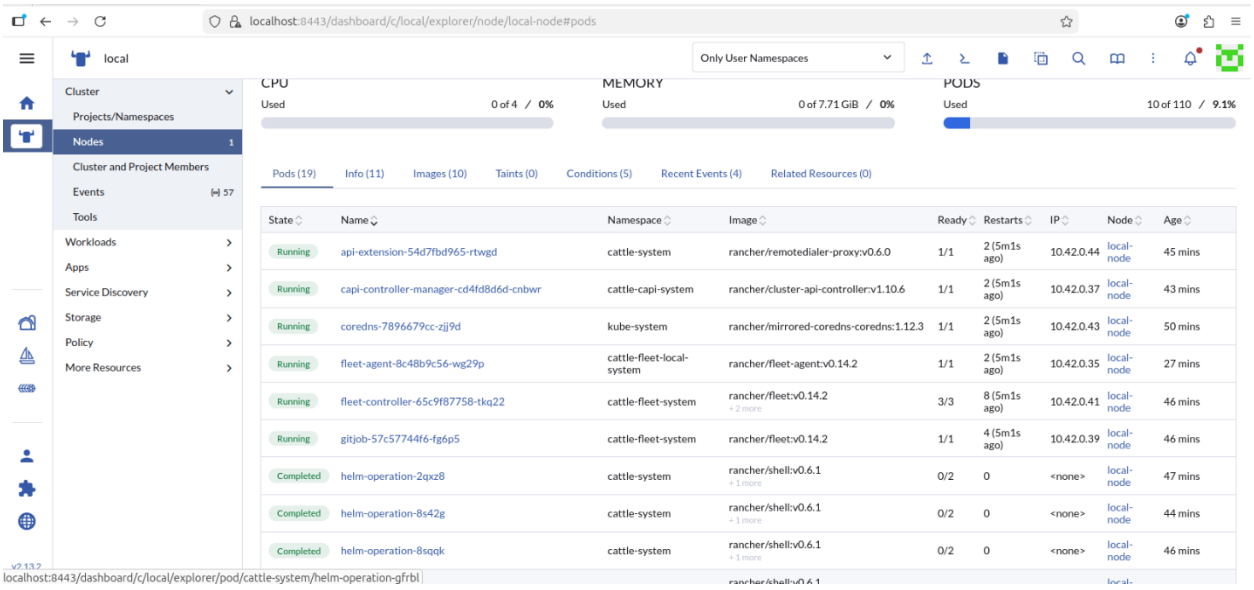
```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods
```

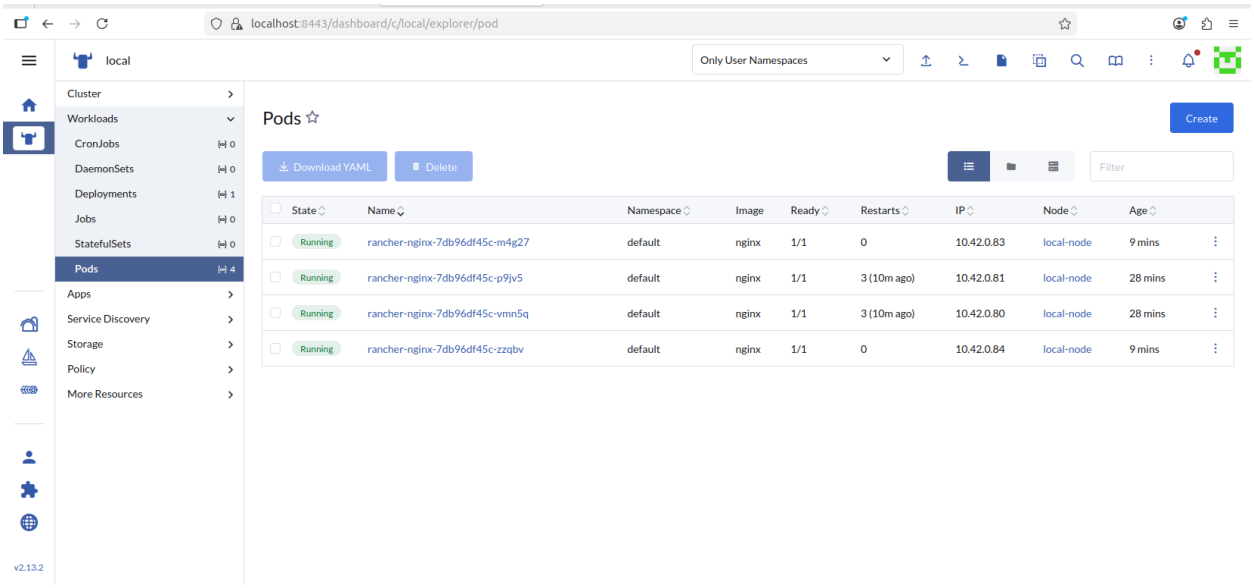
4 pods are seen running. This shows that scaling from the GUI makes real changes to the backend Kubernetes cluster.

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
lab-deployment-787768fbc4-h4dqh     1/1     Running   0          102m
lab-deployment-787768fbc4-rwmqc     1/1     Running   0          102m
lab-deployment-787768fbc4-w2kmn     1/1     Running   0          102m
nginx-dev2                          1/1     Running   0          7h14m
```

Action C: Logs & Exec into Pod

Then go to Pod details and view *View Logs*. Then view *Pods* from *Cluster*.



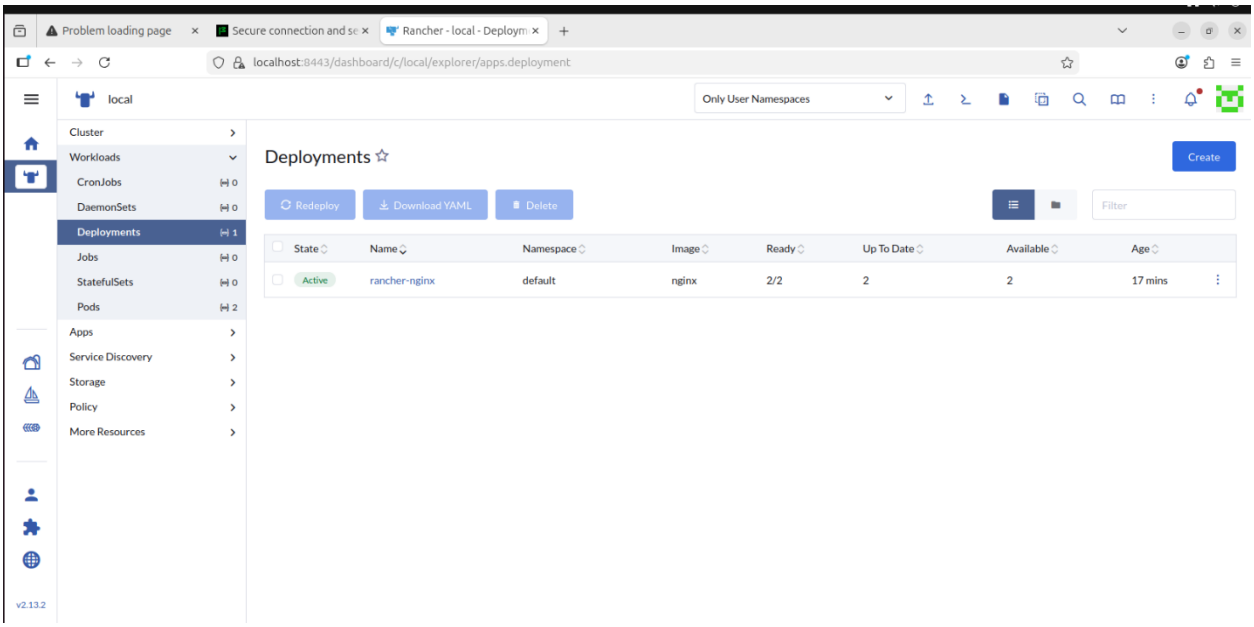


Action D: Edit YAML from Rancher GUI (Rolling Update Proof)

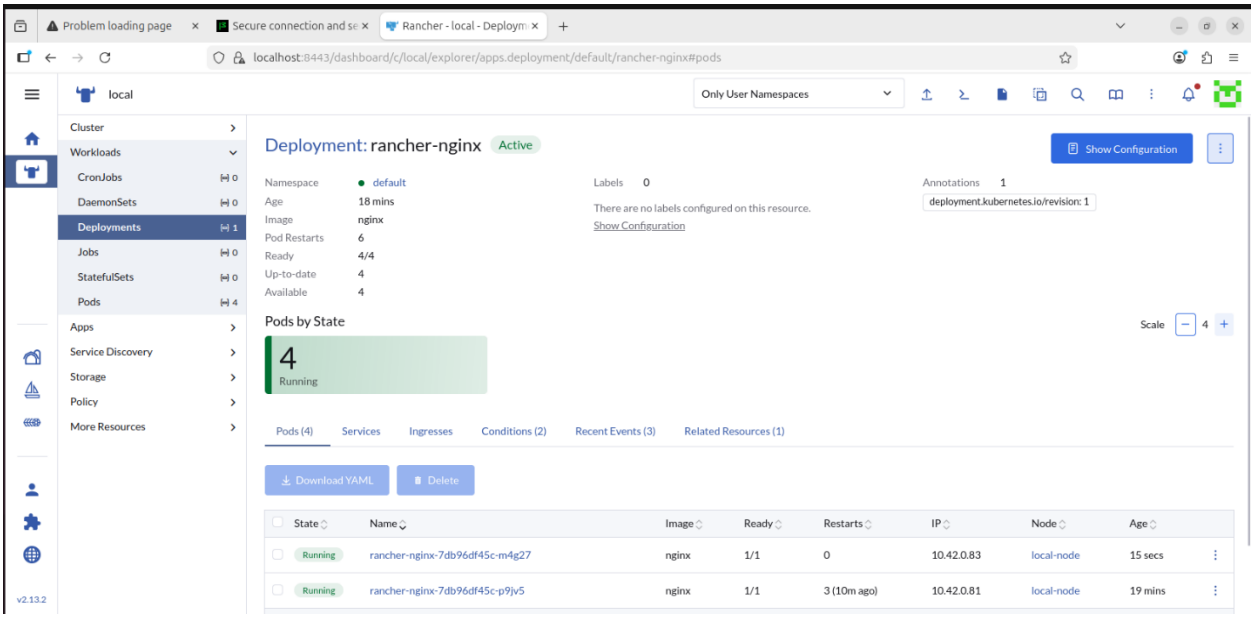
Here is a Rolling Update showing the Deployment update from the GUI.

Editing YAML from the GUI:

Open the Rancher Dashboard, go to Cluster, Workloads, and then Deployments.

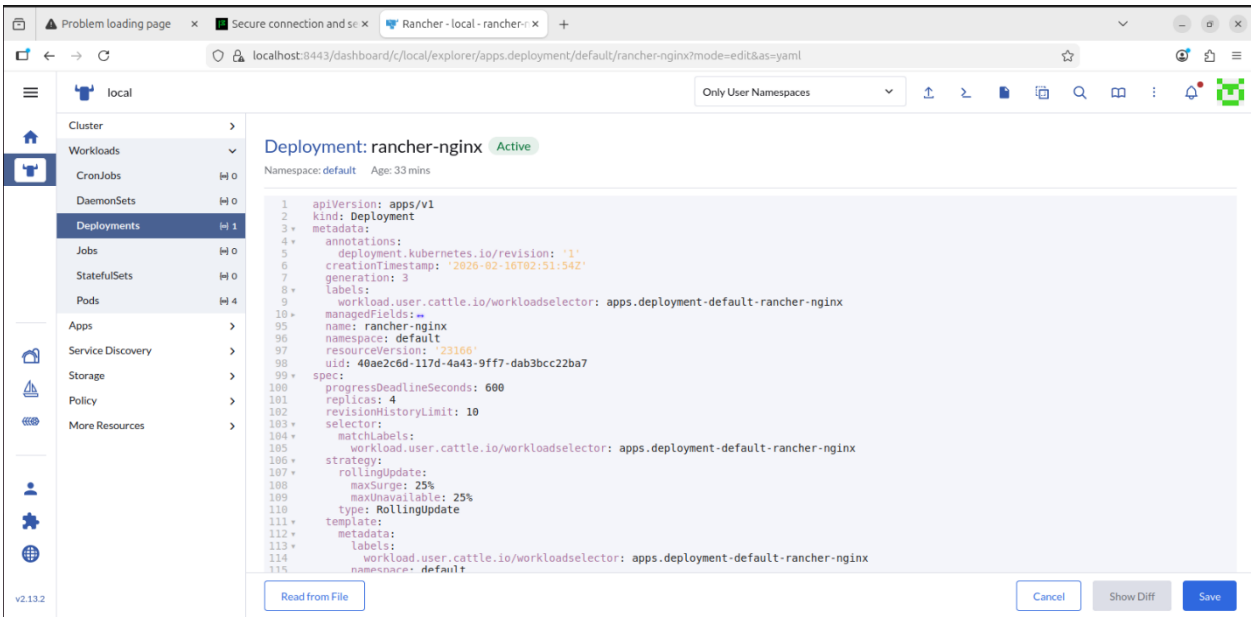


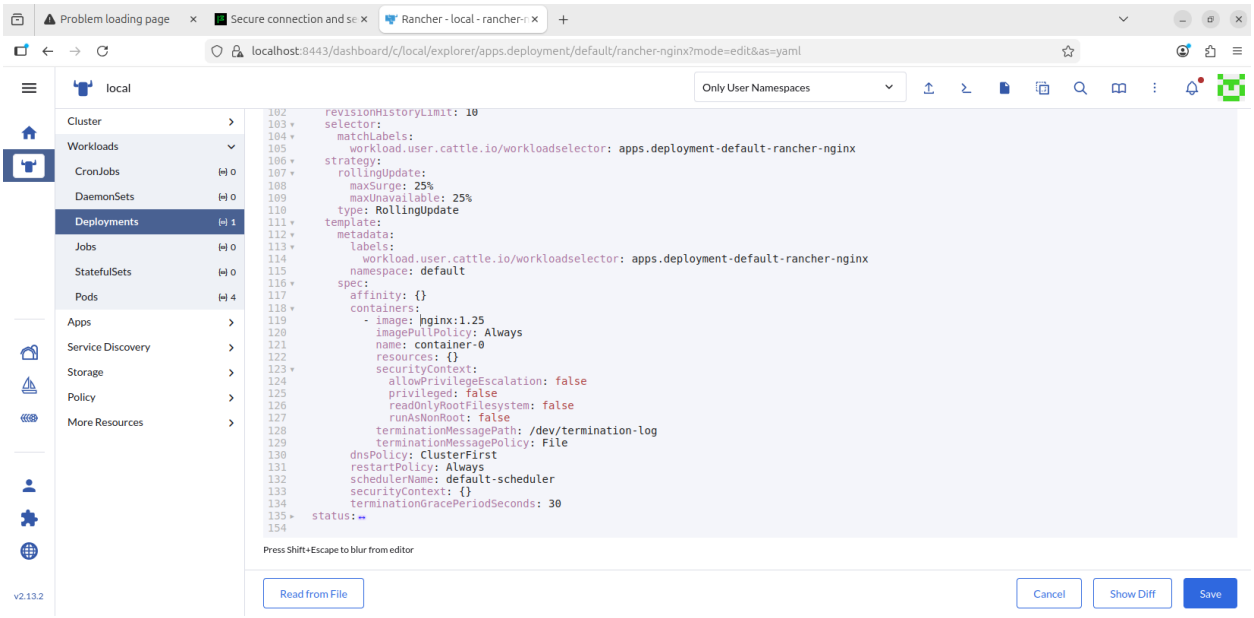
Then Deployment is selected (name: rancher-nginx).



Then go to the *Menu* button on the right and go to *Edit YAML* and change the Image version.

image: nginx:1.21





Changed from *nginx:1.25* to *image: nginx:1.25.*

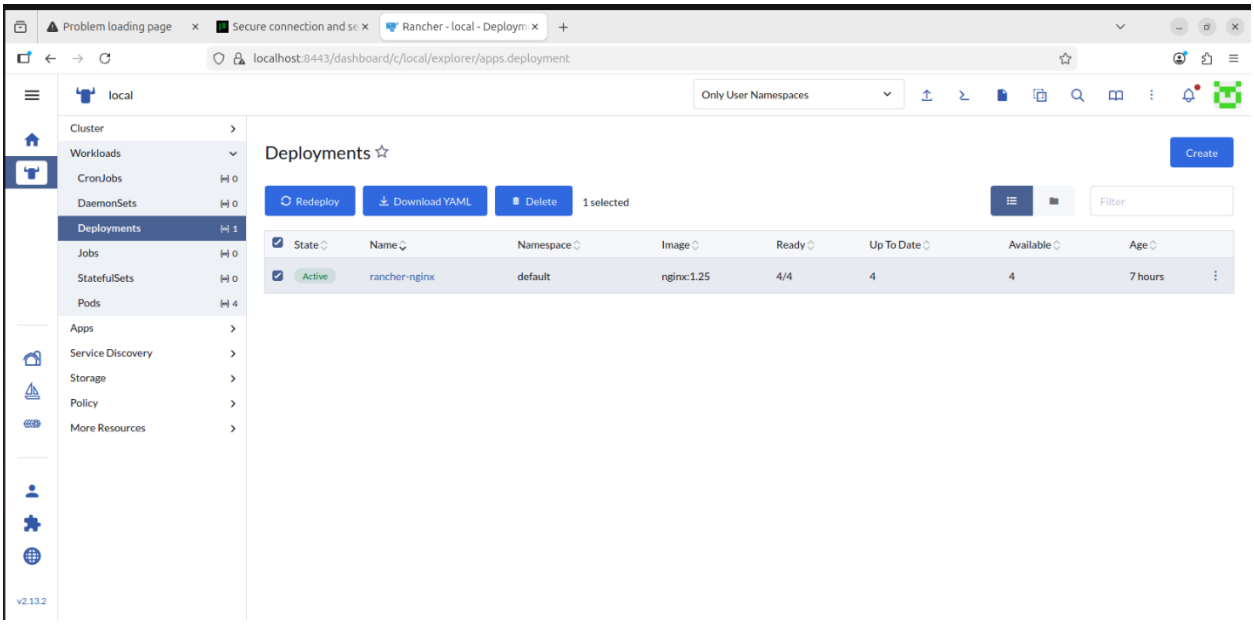


Then Save.

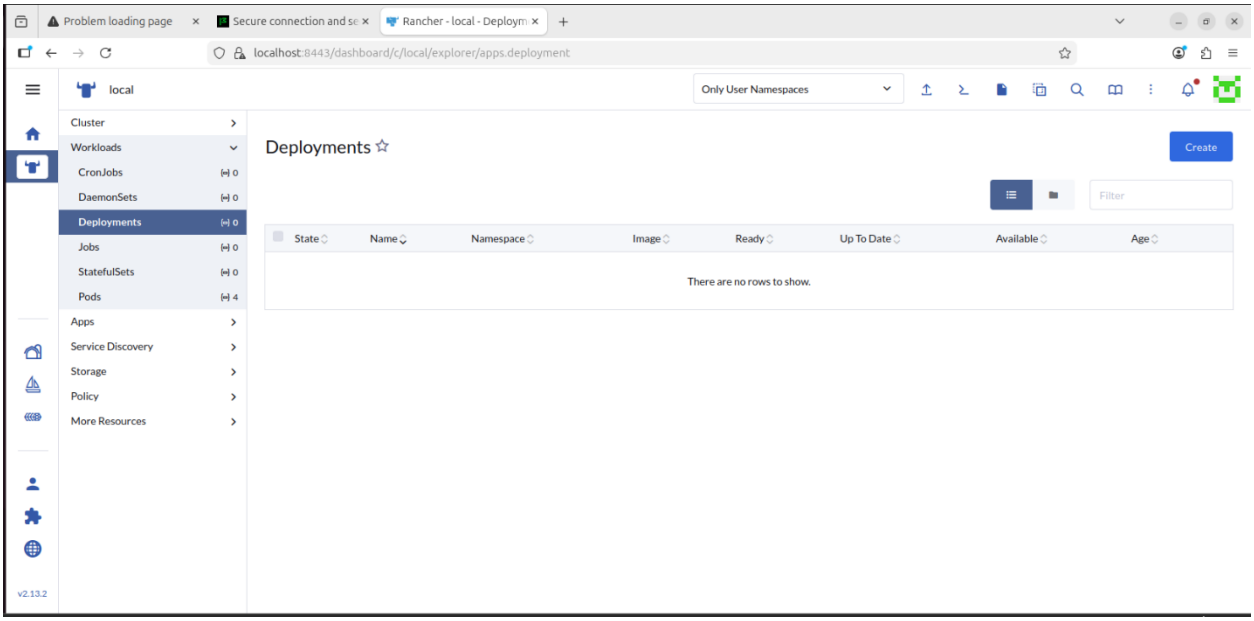
Action 6: Delete Workload from GUI

Delete from GUI:

I went to *Workloads* from Rancher, then *Deployments*, then *lab-deployment.*



Then Delete is confirmed.



CLI Verify:

```
hasib@gitlab-24:~/Desktop/k8s-lab-diamond$ kubectl get pods -l app=lab-app -n default
No resources found in default namespace.
```

The output shows *No resources found* in the default namespace.

References:

[1] NGINX, Inc., "NGINX Open Source," *NGINX Documentation*. Available: https://nginx.org/en/

[3] The Kubernetes Authors, "Kubernetes Documentation," *Kubernetes Official Docs*, 2025. Available: https://kubernetes.io/docs/home/

[3] The Kubernetes Authors, "Kubernetes Components," *Kubernetes Documentation*, 2025. Available: https://kubernetes.io/docs/concepts/overview/components/

[4] The Kubernetes Authors, "Pods," *Kubernetes Documentation*, 2025. Available: https://kubernetes.io/docs/concepts/workloads/pods/

[5] The Kubernetes Authors, "Services, Load Balancing, and Networking," *Kubernetes Documentation*, 2026. Available: https://kubernetes.io/docs/concepts/services-networking/service/

[6] The Kubernetes Authors, "Deployments," *Kubernetes Documentation*, 2025. Available: https://kubernetes.io/docs/concepts/workloads/controllers/deployment/

[7] The Kubernetes Authors, "kubectl Overview," *Kubernetes Documentation*, 2025. Available: https://kubernetes.io/docs/reference/kubectl/

[8] Kubernetes SIG CLI, "Install and Set Up kubectl on Linux," *Kubernetes Documentation*, 2025. Available: https://kubernetes.io/docs/tasks/tools/install-kubectl-linux/

[9] Kubernetes SIG Cluster Lifecycle, "Minikube," *Kubernetes Documentation*, 2026. Available: https://minikube.sigs.k8s.io/docs/

[10] Kubernetes SIG Instrumentation, "Metrics Server," *Kubernetes Documentation*, 2025. Available: https://kubernetes.io/docs/tasks/debug/debug-cluster/resource-metrics-pipeline/

[11] The Kubernetes Authors, "Kubernetes Dashboard," *Kubernetes Documentation*, 2026. Available: https://kubernetes.io/docs/tasks/access-application-cluster/web-ui-dashboard/

[12] Docker Inc., "Docker Engine Overview," *Docker Documentation*. Available: https://docs.docker.com/engine/